JOB SCOUT: AI-POWERED JOB DISCOVERY & ATS OPTIMIZATION

A PROJECT REPORT SUBMITTED TO

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

MASTER OF SCIENCE IN

APPLIED DATA SCIENCE

BY

SHRISH PRASAD CHAKRABORTY (REG. NO. RA2532014010136)

UNDER THE GUIDANCE OF

DR. R. THILAGAVATHY, Ph.D.

Assistant Professor

DEPARTMENT OF COMPUTER APPLICATIONS

FACULTY OF SCIENCE AND HUMANITIES

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

Kattankulathur, 603 203, Chennai, Tamil Nadu

OCTOBER 2026

BONAFIDE CERTIFICATE

This is to certify that the project report titled "**JOB SCOUT: AI-POWERED JOB DISCOVERY & ATS OPTIMIZATION**" is a bona fide work carried out by **SHRISH PRASAD CHAKRABORTY (REG. NO. RA2532014010136)** under my supervision for the award of the Degree of Master of Science in Applied Data Science. To my knowledge, the work reported herein is the original work done by this student.

Dr. R. Thilagavathy	**Dr. P. J. Arul Leena Rose**
Assistant Professor	Professor & Head
Department of Computer Applications	Department of Computer Applications
(GUIDE)	

INTERNAL EXAMINER	**EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

I am grateful to everyone who supported me during the completion of this project.

I would like to thank my parents for their steady encouragement and support throughout my studies.

I thank **Dr. T. R. Paarivendhar**, Chancellor of SRM Institute of Science & Technology, for providing an academic environment in which I could pursue this work.

I am grateful to **Dr. A. Duraisamy**, Dean of the Faculty of Science and Humanities, for encouraging students to explore new ideas and projects.

I thank **Dr. S. Albert Antony Raj**, Deputy Dean of the Faculty of Science and Humanities, for his guidance and support during the project.

I thank **Dr. P. J. Arul Leena Rose**, Professor and Head of the Department of Computer Applications, for her support throughout the programme.

My sincere thanks go to my project guide, **Dr. R. Thilagavathy**, Assistant Professor in the Department of Computer Applications, for her guidance, suggestions and feedback during the development of Job Scout.

I also thank the department faculty members for their comments and suggestions during the project reviews.

Finally, I thank my friends and everyone else who helped me complete this work.

Shrish Prasad Chakraborty

TABLE OF CONTENTS

ABSTRACT ..... **x**

LIST OF TABLES ..... **xi**

LIST OF FIGURES ..... **xii**

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE ..... **xiii**

Chapter 1 - Introduction 1

1.1 Background of the Study 1

1.2 Problem Statement 1

1.3 Objectives of the Project 2

1.4 Scope of the Project 2

1.5 Significance of the Project 3

Chapter 2 - Literature Review and Existing Systems 4

2.1 Review of Existing Systems and Research 4

2.2 Comparative Analysis 7

2.3 Research Gap and Limitations of Existing Systems 8

2.4 Proposed System and Approach 9

Chapter 3 - Software Requirements Analysis 10

3.1 Hardware Specifications 10

3.2 Software Specifications 10

3.3 Software Tools and Their Features 11

Chapter 4 - System Analysis 13

4.1 Requirements Specification 13

4.2 Characteristics of the Existing System 13

4.3 Proposed System 14

4.4 Feasibility Study 15

4.5 Software Requirements Specification 15

Chapter 5 - System Design 19

5.1 System Architecture and Overall Workflow 19

5.2 Context Diagram 22

5.3 Use Case Diagram 22

5.4 Activity Diagram 23

5.5 Data Flow and Entity-Relationship Diagrams 24

5.6 Database Design....27
5.7 User Interface Design....29
5.8 Module Design....31
5.9 API Design....34
5.10 Security Design....36
Chapter 6 - System Implementation....38
6.1 Module Descriptions....38
6.2 Data Collection and Dataset Description....39
6.3 Data Preprocessing....39
6.4 Feature Representation and Extraction....40
6.5 Model Architecture and Algorithm Design....40
6.6 System and Model Implementation....41
6.7 Validation Checks....45
6.8 User Interface Implementation....46
Chapter 7 - Testing and Evaluation....47
7.1 Test Cases....47
7.2 Unit Testing....48
7.3 Integration Testing....49
7.4 System Testing....50
7.5 Model and Performance Evaluation....50
7.6 Performance Metrics....59
7.7 Comparative Analysis....60
Chapter 8 - Results and Conclusion....61
8.1 Results....61
8.2 Discussion and Analysis....62
8.3 Limitations....63
8.4 Lessons Learnt....64
8.5 Conclusion....65
8.6 Future Enhancements....65

Chapter 9 - Appendices.....67
Appendix 1 - Plagiarism Certificate.....67
Appendix 2 - Screenshots.....68
Appendix 3 - Sample Coding.....72
Appendix 4 - User Documentation.....75
Appendix 5 - Glossary.....76
Chapter 10 - References.....77

ABSTRACT

Job Scout is a self-hosted platform that helps users find job vacancies suited to their resumes, review each match, and prepare application materials.

The project addresses two practical issues. Keyword-based filters can miss relevant experience when candidates describe the same skill in different words. In addition, online resume tools may process personal documents on provider-operated systems, depending on the service and its settings. Job Scout collects listings from five job portals using Playwright and Celery. It converts resumes and job descriptions into embeddings with all-MiniLM-L6-v2 and stores and searches those vectors in PostgreSQL using pgvector and HNSW. The ranking combines semantic similarity (70%) with keyword matching (30%). An ATS diagnostic highlights missing keywords, and a locally hosted Llama-3.2-3B-Instruct model uses retrieval-augmented generation (RAG) to help draft resume bullet points and cover letters. FastAPI, Next.js, Redis and Docker Compose connect the API, interface, background tasks and deployment.

In the recorded CPU-only tests, raw vector queries over 100,000 postings averaged 2.88 ms, while the two-stage hybrid query averaged 9.08 ms through the API; its 95th-percentile latency was 11.13 ms. The automated suite passed 126 tests, with 79% backend code coverage. On the synthetic retrieval benchmark, hybrid ranking reached an NDCG@10 score of 0.985, compared with 0.896 for keyword-only ranking. The evaluation also found that 9.5% of bullets generated by Llama-3.2-3B-Instruct mentioned a technology absent from the resume. Because the retrieval labels were synthetic rather than human-labelled examples from real job searches, these measurements should be treated as an initial evaluation. The tests show that the main workflow can run locally; retrieval quality on real job data and the factual accuracy of generated content need further evaluation.

Keywords: Retrieval-Augmented Generation, Resume Parsing, Artificial Intelligence, Job Matching, Vector Search.

LIST OF TABLES

No.		Page No.
Table 2.1	Comparative Analysis of Existing Tools and Job Scout	7
Table 3.1	Hardware and Software Requirements	10
Table 3.2	Software and Tools Used in Job Scout	11
Table 4.1	Functional Requirements	15
Table 4.2	Non-Functional Requirements	17
Table 5.1	Database Schema Definitions	28
Table 5.2	Registered Application Endpoints	35
Table 6.1	Modules and Their Implementation Artefacts	38
Table 6.2	Job Data Sources and Scrapers	39
Table 6.3	Validation Checks Implemented in Job Scout	45
Table 7.1	Test Strategy	47
Table 7.2	Measured Backend Code Coverage	47
Table 7.3	Unit Test Cases and Results	48
Table 7.4	Integration Test Cases and Results	49
Table 7.5	System and Security Test Cases and Results	50
Table 7.6	Vector Search Latency in Milliseconds	51
Table 7.7	HNSW Parameter Sweep, 10,000 Rows	52
Table 7.8	HNSW Parameter Sweep, 100,000 Rows	52
Table 7.9	Single-Stage and Two-Stage Hybrid Query in the Benchmark Script	54
Table 7.10	End-to-End Latency of POST /jobs/matches	54
Table 7.11	Retrieval Quality of the Ranking Strategies	56
Table 7.12	Sweep of the Vector Weight α	57
Table 7.13	Generation Quality and Grounding	58
Table 7.14	Summary of Measured Metrics	59
Table 7.15	Comparison of Baseline and Optimised Configurations	60
Table 8.1	Objectives and Outcomes	62
Table 9.1	Key Implementation Files	72
Table 9.2	Glossary of Terms	77

LIST OF FIGURES

No.		Page No.
Fig. 5.1	Overall System Architecture Diagram	19
Fig. 5.2	Service-Level Architecture of the Dockerised Deployment	21
Fig. 5.3	Hybrid RAG Pipeline Workflow	21
Fig. 5.4	Context Diagram (Level 0)	22
Fig. 5.5	Use Case Diagram of Job Scout	23
Fig. 5.6	Activity Diagram of the Candidate Workflow	24
Fig. 5.7	Data Flow Diagram (Level 1)	25
Fig. 5.8	Conceptual Entity-Relationship Diagram	26
Fig. 5.9	UI Wireframe of the Job Discovery View	30
Fig. 5.10	UI Wireframe of the AI Copilot View	30
Fig. 5.11	UI Wireframe of the Kanban Board View	31
Fig. 9.1	Job Discovery View (Best Matches)	69
Fig. 9.2	Resume Vault	69
Fig. 9.3	Job Tracker (Kanban Board)	70
Fig. 9.4	AI Copilot View	70
Fig. 9.5	Neural Profile View	71
Fig. 9.6	Market Intelligence View (Market Radar)	71
Fig. 9.7	System Configuration Dialog	72
Fig. 9.8	Additional Interface View	72

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

ANN	Approximate Nearest Neighbour
API	Application Programming Interface
ATS	Applicant Tracking System
BERT	Bidirectional Encoder Representations from Transformers
bfloat16	16-bit brain floating-point format
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
DFD	Data Flow Diagram
ER	Entity-Relationship
GPU	Graphics Processing Unit
HNSW	Hierarchical Navigable Small World
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
MRR	Mean Reciprocal Rank
NDCG	Normalized Discounted Cumulative Gain
NFR / FR	Non-Functional Requirement / Functional Requirement
NLP	Natural Language Processing
ORM	Object-Relational Mapping
P@k	Precision among the top k results
R@k	Recall among the top k results
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RSS	Resident Set Size (process memory measure)
SBERT	Sentence-BERT
SQL	Structured Query Language
TF-IDF	Term Frequency–Inverse Document Frequency
UML	Unified Modeling Language
VRAM	Video Random Access Memory

Chapter 1 - Introduction

1.1 Background of the Study

Applicant tracking systems (ATS) help employers collect and manage applications and search candidate information. Some screening workflows rely heavily on keywords or predefined criteria. As a result, relevant experience can be harder to identify when a candidate describes a skill using terms that differ from those in a job description.

1.2 Problem Statement

Candidates may have relevant technical and interpersonal skills yet receive low match scores when their resumes use terminology that differs from a job description. Tailoring a resume for each application can also take time and requires careful comparison of the role requirements with the candidate's experience. Online AI resume tools may process personal documents on provider-operated systems; data handling depends on each service and its settings. These concerns motivate a locally deployable tool that supports job matching and resume preparation while keeping candidate documents on the user's machine.

The limitations of the existing recruitment workflow can be summarised as three interrelated problems:

- Limited transparency in automated screening: studies of algorithmic hiring tools have found that vendors may provide limited information about how their systems are developed, validated and audited for bias [1].
- Privacy and model memorisation: research has shown that large language models can reproduce parts of their training data, including sensitive information [2]. This is one reason to review data handling carefully when using AI services; it does not establish how any particular resume service stores or uses submitted documents.
- **Workflow fragmentation:** job discovery, ATS diagnosis, tailoring and application tracking are spread across separate boards, tools and spreadsheets.

1.2.1 Proposed Solution

To address these issues, this project introduces Job Scout, an AI-assisted job discovery and ATS optimisation platform. It collects job postings, compares a candidate's resume with job descriptions using semantic and keyword signals, and uses a locally hosted language model to help draft application content. The system uses PyTorch for local model inference and PostgreSQL with pgvector for vector storage and HNSW-based approximate nearest-neighbour search. The design aims to keep candidate documents on the host machine; the offline test in Chapter 7 checks the core processing workflow with external network access blocked.

1.3 Objectives of the Project

The primary objectives of this project are as follows:

- To build an automated pipeline for collecting job listings using Playwright and Celery-Beat scheduling.
- To develop a hybrid job-matching engine using pgvector, with a target mean query latency below 10 ms on the tested 100,000-row benchmark.
- To integrate locally hosted language models through Hugging Face Transformers and PyTorch for drafting resume bullet points and cover letters.
- To build a responsive web application with FastAPI for backend processing and Next.js for the frontend.

1.4 Scope of the Project

The project covers job listing collection, resume processing, vector-based job matching, ATS keyword diagnostics and local language-model generation. The current implementation focuses on technology-related roles, including software engineering and data science. Docker Compose is used to run the system locally without a dependency on commercial AI APIs. Initial setup and job scraping still require network access to obtain dependencies, model files and listings.

The current version does not target non-technical roles, non-English resumes or concurrent multi-user deployment; these are discussed in Chapter 8. It does not submit applications automatically. Candidates review the results and remain responsible for

checking each job portal’s terms and conditions before using its data or submitting an application.

1.5 Significance of the Project

Job Scout is intended to reduce the manual effort involved in job discovery and resume tailoring. Its ATS diagnostic identifies missing keywords and terminology differences for the candidate to review when preparing an application.

The project also documents practical design trade-offs, including model size versus inference speed, hybrid versus purely semantic ranking, and fallback behaviour when the preferred model cannot be loaded. These trade-offs were measured on CPU-only hardware; GPU generation performance was not evaluated.

Chapter 2 - Literature Review and Existing Systems

2.1 Review of Existing Systems and Research

2.1.1 Evolution and Limitations of Applicant Tracking Systems

Applicant tracking systems began as digital tools for collecting and organising job applications and now include broader features for managing recruitment workflows. Platforms such as Workday, Greenhouse, iCIMS and Taleo are used by many employers. In its 2026 report, Jobscan detected an ATS for 97.4% of Fortune 500 companies [3]. Bogen and Rieke [4] examine how algorithmic tools are used in the recruitment process.

Some resume-screening workflows use keywords or stated criteria to narrow a candidate pool. When the wording in a resume differs from the wording in a job description, relevant experience may be harder to find. In the Hidden Workers report, Fuller et al. [5] found that 88% of surveyed employers believed qualified high-skills candidates were screened out because their resumes did not match the exact criteria in a job description.

Raghavan et al. [1] found that vendors disclosed limited information about the development and validation of algorithmic hiring assessments, including their approaches to bias auditing. This research supports the need for transparency and careful evaluation when such tools influence recruitment decisions.

2.1.2 Natural Language Processing for Job and Candidate Matching

Natural language processing (NLP) has been applied to job and candidate matching through methods ranging from classical information retrieval to contextual language models. Early approaches included Term Frequency–Inverse Document Frequency (TF-IDF) and Latent Semantic Analysis (LSA) [6]. These methods provide useful document representations but have limitations in modelling contextual meaning and the different senses of a word.

Transformer-based models improved contextual text representation. Devlin et al. [7] introduced BERT (Bidirectional Encoder Representations from Transformers), which learns context-sensitive representations from surrounding text. Reimers and Gurevych [8] introduced Sentence-BERT (SBERT), which adapts BERT-style models to produce

sentence embeddings suited to similarity comparisons. Job Scout uses all-MiniLM-L6-v2, which produces 384-dimensional embeddings and offers a practical balance between embedding size and inference cost [9].

For resume-to-job matching, Qin et al. [10] proposed the Ability-aware Person–Job Fit Neural Network (APJFNN), which uses hierarchical attention to represent job requirements and candidate experience. Qin et al. [11] later developed an enhanced neural approach to person–job fit. In the related field of passage retrieval, Karpukhin et al. [12] showed that dense retrieval can outperform BM25 on open-domain question answering. Those results do not directly validate resume matching, but they support evaluating contextual embeddings alongside lexical signals.

2.1.3 Vector Databases and Approximate Nearest Neighbour Search

Dense embeddings require storage and retrieval methods that can compare high-dimensional vectors efficiently. Standard relational indexes are designed mainly for scalar fields and exact-match conditions, so vector retrieval generally needs dedicated operators and index support. PostgreSQL extensions such as pgvector add these capabilities to a relational database.

Malkov and Yashunin [13] introduced the Hierarchical Navigable Small World (HNSW) graph for approximate nearest-neighbour search. HNSW organises vectors in a multi-layer graph and can provide fast retrieval while trading some recall for lower query latency. Independent benchmarks compare HNSW with other approximate nearest-neighbour methods across different datasets and configurations [14].

Dedicated vector-search systems include Pinecone, Weaviate, Milvus and Qdrant. For applications already using PostgreSQL, pgvector [15] offers vector storage, distance operators and IVFFlat and HNSW indexes within the same database. For this single-user deployment, keeping relational data and embeddings together simplifies the architecture and avoids operating a separate vector database service.

2.1.4 Retrieval-Augmented Generation for Personalised Content Generation

Large language models can produce fluent text but may introduce unsupported details when asked to write about an individual’s work history or skills. Retrieval-augmented generation (RAG), described by Lewis et al. [16], supplies a model with relevant context

retrieved at inference time. In a typical RAG pipeline, a query is used to retrieve relevant documents or document chunks; the retrieved material is then included with the prompt. This gives the model evidence to use, but does not guarantee that every generated statement will be accurate.

Shuster et al. [17] found that retrieval augmentation can reduce hallucinations in knowledge-grounded dialogue, while Gao et al. [18] survey RAG methods and their applications. For resume assistance, retrieved or parsed resume information can help keep generated bullet points and cover letters tied to the candidate's stated experience. The output still requires review because the model may add unsupported information.

RAG quality depends in part on the context supplied to the model. Izacard and Grave [19] showed that the passages provided to a reader model affect answer quality, while longer contexts also increase computation. This motivates testing retrieval settings such as the number of candidates selected and the similarity criteria used by the application.

2.1.5 Privacy-Preserving Local AI Deployment

Online AI services process submitted content according to their own service design and data-handling policies, so users should review those policies before uploading resumes or other personal documents. Carlini et al. [2] demonstrated that language models can memorise and reproduce examples from their training data; that finding is evidence of a model risk, not proof that a particular hosted resume service stores or trains on user uploads. Running inference locally can reduce the need to transmit candidate documents to an external AI provider.

Open-weight model families, including Meta's Llama 2 [20] and Llama 3 [21], Mistral 7B [22] and Google's Gemma [23], make local model deployment possible subject to their individual licences and system requirements. Meta introduced Llama 3.2 models for edge and on-device use [24]. A locally hosted model can generate structured text without sending the prompt to a remote inference API, provided the application and its dependencies are configured and tested accordingly.

Frameworks such as Hugging Face Transformers [25], llama.cpp [26] and vLLM [27] support local model inference. Job Scout runs unsloth/Llama-3.2-3B-Instruct locally through PyTorch and Hugging Face Transformers. The offline test described in Chapter 7

completed resume parsing, embedding, matching and generation while external network access was blocked; this supports the offline claim for those tested steps, rather than proving that every possible application path is free from data egress.

2.1.6 Task Queues for Automated Data Collection

Collecting job listings from multiple web sources requires scheduled background tasks and a way to retry or monitor work independently of user requests. Celery is a widely used Python task queue [28]. When paired with Redis as a broker, it can dispatch asynchronous tasks and support periodic scheduling through Celery-Beat. Job Scout uses this pattern to run scraping tasks outside the main API request path.

Playwright [29] automates browser interactions and can render JavaScript-heavy pages, which is useful when job details are loaded dynamically. Job Scout combines Playwright with Celery-Beat to schedule collection tasks and uses a separate worker to process them. Portal structures and terms can change, so the scrapers require maintenance and must be used in accordance with each site's rules.

2.2 Comparative Analysis

Table 2.1 compares Job Scout with the commercial tools discussed in the literature survey, using the characteristics of those tools as described in this chapter and in Section 4.2. The comparison is qualitative: it contrasts the design of each system rather than benchmarking their output.

Table 2.1 Comparative Analysis of Existing Tools and Job Scout

Characteristic	Jobscan	Teal	Rezi	Job Scout
Resume–job matching	ATS Match Rate and keyword scanner	Job targeting and keyword matching	Job-description targeting and Rezi Score	Hybrid matching: 70% semantic (pgvector HNSW) + 30% keyword
AI content generation	AI Resume Builder and writing coach	AI-assisted writing, including Bullet Coach	AI Resume Agent and writing tools	Yes (RAG with a local LLM)

Characteristic	Jobscan	Teal	Rezi	Job Scout
Where AI processing runs	Online service; processing location not independently verified	Online service; processing location not independently verified	Online service; processing location not independently verified	Local inference in the tested setup
Candidate-data handling tested in this report	Not tested independently	Not tested independently	Not tested independently	Core workflow passed an offline test
Equivalent self-hosted workflow documented publicly	Not found in reviewed sources	Not found in reviewed sources	Not found in reviewed sources	Yes (Docker Compose)
Cost model	Commercial service; plan varies	Commercial service; plan varies	Commercial service; plan varies	No paid AI API used in the tested setup; hardware and power still cost money

2.3 Research Gap and Limitations of Existing Systems

The reviewed literature covers several relevant areas, including algorithmic hiring, person–job matching, vector search, local language-model deployment and RAG-based generation. The sources reviewed for this report discuss these areas largely as separate problems. Job Scout brings them together in one self-hosted prototype that supports job discovery, semantic matching, ATS keyword analysis, local content drafting and application tracking. This report presents that integration as the project's engineering contribution; it does not claim that no similar system has ever been developed.

The commercial tools considered here address different parts of the job-search workflow. This qualitative comparison uses Jobscan's 2026 ATS Usage Report and the public product pages for Teal and Rezi [3], [30], [31]. The report contrasts advertised features; ranking quality, internal architecture and data-handling practices were not independently tested. The public pages reviewed for this project did not document an equivalent local, self-hosted deployment, although product features and service policies may change.

Job Scout implements a self-hosted workflow that combines pgvector-based semantic matching, locally generated text using resume context, scheduled job discovery and an

interactive Next.js frontend. It can be deployed on a single machine without relying on a commercial AI inference API. Each dependency and model remains subject to its own licence and usage conditions.

2.4 Proposed System and Approach

Job Scout uses a six-stage workflow. First, Celery-Beat schedules Playwright scrapers to collect listings from five job portals and create 384-dimensional embeddings. Second, the candidate's resume is parsed, converted to Markdown and embedded with the same all-MiniLM-L6-v2 model. Third, PostgreSQL combines vector similarity with keyword evidence to rank job candidates. Fourth, an ATS diagnostic reports missing terms for a selected job. Fifth, a local Llama-3.2-3B-Instruct model uses parsed resume information to help draft resume bullets and cover letters; the company-research feature uses internet search snippets as context for a separate summary. Finally, a Kanban board records application progress.

The application runs in a Docker Compose deployment, and model downloads are disabled at inference time after the required model files have been prepared locally. The core processing workflow was tested with external network access blocked. Scheduled job scraping requires internet access, and initial setup requires access to dependencies and model files unless they have already been downloaded. The architecture and implementation are described in Chapters 5 and 6.

Chapter 3 - Software Requirements Analysis

3.1 Hardware Specifications

The minimum and recommended hardware specifications are summarised in Table 3.1.

Table 3.1 Hardware and Software Requirements

Component	Minimum (CPU-only inference)	Recommended (GPU-accelerated inference)
Processor	4-core CPU (Intel i5 / AMD Ryzen 5 class)	6-core CPU or better
RAM	16 GB (all results were measured on a 15 GiB machine)	16 GB or more
GPU (optional)	None; CPU inference with BFloat16	NVIDIA GPU with CUDA support (not benchmarked in this project)
Storage	20 GB free disk space (SSD)	30 GB free disk space (SSD or NVMe)
Operating system	Ubuntu 22.04 / Windows 11 / macOS	Ubuntu 22.04 LTS (Docker host)
Measured generation speed	1.82 tokens per second (129 tokens in 70.79 s)	Not measured (no GPU available)

Minimum configuration (CPU-only inference): 16 GB RAM, a 4-core CPU and 20 GB of free disk space. On the tested 12-thread Intel Core i7-1255U system, the model generated 1.82 tokens per second and the backend reached a peak resident memory of 7.06 GB (Section 7.5.3). The measured workload, together with PostgreSQL, Redis and the other services, is not recommended for an 8 GB machine.

Recommended configuration (GPU-accelerated inference): at least 16 GB RAM, a 6-core CPU, an NVIDIA GPU with CUDA support and 30 GB of free disk space. The CUDA code path using float16 is implemented, but GPU generation speed was not measured in this project.

3.2 Software Specifications

The software requirements for developing and deploying Job Scout are listed below:

- **Operating System:** Any OS supporting Docker (Linux, macOS, Windows 10/11 with WSL2).
- **Container Runtime:** Docker Engine 20.10+ with Docker Compose V2.
- **Backend Runtime:** Python 3.11 (python:3.11-slim image) with the versions pinned in requirements.txt: FastAPI 0.139.2, SQLAlchemy 2.0.51, PyTorch 2.13.0, Transformers 5.14.1 and Sentence-Transformers 5.6.0.
- **Frontend Runtime:** Node.js 20 (node:20-alpine image) with Next.js 16.2.7, React 19.2.4 and Tailwind CSS 4.
- **Database:** PostgreSQL 15 with the pgvector extension (provided by the ankane/pgvector Docker image; Python client pgvector 0.5.0).
- **Message Broker:** Redis 7+ (Alpine).
- **Task Queue:** Celery 5.6.3 with Celery-Beat.
- **Browser Automation:** Playwright for Python 1.61.0.

3.3 Software Tools and Their Features

Job Scout uses a stack centred on open-source software and locally available models. Table 3.2 lists each tool, its role and the features used in the project. Model weights and dependencies have separate licences and usage terms; these should be checked before redistribution or commercial use. The relevant documentation is listed in References [28], [29], and [32]–[38].

Table 3.2 Software and Tools Used in Job Scout

Tool	Role in Job Scout	Key features used
FastAPI	Backend REST API	Asynchronous I/O, Pydantic request validation, automatic OpenAPI documentation, StreamingResponse for token streaming
Next.js 16 / React 19	Frontend web application	Component-based UI, server-side rendering capability, token-by-token streaming updates
PostgreSQL with pgvector	Relational and vector storage	Vector(384) column type, cosine distance operator (<=>), HNSW index
Redis	Message broker and response cache	Celery task dispatch and result storage, FastAPI response cache

Tool	Role in Job Scout	Key features used
Celery and Celery-Beat	Task queue and scheduler	Asynchronous scraping, periodic task scheduling, retry on failure
Playwright	Browser automation for scraping	Rendering of JavaScript-heavy job portal pages
PyTorch and HuggingFace Transformers	Local model inference	Text-generation pipeline; BFloat16 on CPU and Float16 on CUDA
all-MiniLM-L6-v2	Embedding model	384-dimensional sentence embeddings for resumes and jobs
Llama-3.2-3B-Instruct (TinyLlama fallback)	Local language model	Resume bullet, cover letter and company research generation
Docker Compose	Container orchestration	Six services (db, redis, backend, scraper, celery-beat, frontend) as one deployable unit

Chapter 4 - System Analysis

4.1 Requirements Specification

The requirements were derived from the problem statement in Chapter 1 and the workflow limitations discussed in Section 4.2. The Candidate is the primary human actor, and the Celery-Beat Scheduler triggers automated scraping tasks. The requirements cover four areas: job listing ingestion and indexing, resume parsing and embedding, job matching and ATS diagnostics, and generation and tracking of application materials.

The detailed functional requirements (FR-01 to FR-15) and non-functional requirements (NFR-01 to NFR-08) are specified in Section 4.5, and the use cases are analysed in Section 4.5.3.

4.2 Characteristics of the Existing System

The current job-search workflow may require candidates to visit several portals, scan listings and compare each role with their experience. For every promising role, the candidate must review the job description and decide which parts of the resume to emphasise. Repeating these steps across many applications takes time and can lead to mistakes.

Commercial tools cover different parts of this workflow. Jobscan provides resume-to-job keyword scoring, while Teal and Rezi offer resume-building and AI-assisted writing features. The exact capabilities and data-handling practices depend on each product and its configuration; this report does not independently audit those services. The tools reviewed here do not combine the same locally deployed job collection, matching, diagnostics, content drafting and application tracking workflow as Job Scout.

The fundamental limitations of the existing system can be summarised as follows:

- Manual job discovery across multiple portals, leading to significant time overhead and missed opportunities.
- Screening based heavily on keyword overlap can miss related skills when the resume and job description use different terms.

- AI processing through external services can reduce the user’s control over where resume content is handled, making the provider’s data practices and settings important considerations.
- No integrated application tracking, forcing candidates to maintain separate spreadsheets or external tools.
- No feedback loop between ATS diagnostic results and AI content generation, requiring manual interpretation and re-drafting.

4.3 Proposed System

Job Scout is designed as a self-hosted platform for job discovery and ATS optimisation. It brings five main subsystems into one application:

- **Automated Job Discovery Engine:** a scheduled Playwright scraping pipeline coordinated by Celery-Beat collects listings from LinkedIn, Indeed, Naukri, RemoteOK and We Work Remotely. Each listing is embedded into a 384-dimensional vector with all-MiniLM-L6-v2 and stored in PostgreSQL using pgvector.
- **Semantic Matching Engine:** A cosine similarity-based ranking system that compares the candidate's resume embedding against all stored job embeddings using pgvector's HNSW index, returning a ranked list of semantically relevant openings with low query latency for the pure vector query (2.88 ms on average at 100,000 job rows with ef_search = 400).
- **ATS Keyword Diagnostic Engine:** compares a candidate’s resume with a selected job description, identifies missing keywords and reports a score with recommendations for resume improvement.
- **Local LLM Generation Engine:** uses a retrieval-augmented generation (RAG) pipeline with unsloth/Llama-3.2-3B-Instruct through local inference to help draft resume bullet points and cover letters from parsed resume information. Generated text can contain unsupported details and should be reviewed before use.
- **Application Tracking Kanban Board:** lets candidates track applications through five stages: Interested, Applied, Interviewing, Offered and Rejected. It stores status changes and optional notes.

4.4 Feasibility Study

4.4.1 Technical Feasibility

The system uses FastAPI, Next.js, PostgreSQL with pgvector, Redis, Celery and Docker Compose. These components are widely used and have documentation and community support. However, the licence terms differ across the software dependencies and model weights, so each component must be considered separately before redistribution or commercial deployment.

4.4.2 Operational Feasibility

The system is designed for single-user, self-hosted use. Docker Compose coordinates the services, while the launcher script handles several setup steps. Once dependencies and model files are present, resume parsing, matching, AI inference and application tracking can run locally; scheduled scraping needs internet access. The offline test in Chapter 7 verifies a defined core workflow with network egress blocked.

4.4.3 Economic Feasibility

The tested configuration does not require paid AI API subscriptions or a cloud-hosting account. Users still need suitable hardware, electricity and storage, and they may incur costs for maintaining the machine. CPU-mode testing used a machine with 15 GiB of RAM; the report recommends 16 GB or more for local deployment.

4.5 Software Requirements Specification

4.5.1 Functional Requirements

The functional requirements of the Job Scout system are enumerated as follows:

Table 4.1 Functional Requirements

ID	Requirement
FR-01	The system shall allow users to upload resumes in PDF, DOCX, DOC, MD, or TXT format, with a maximum file size of 10 MB.
FR-02	The system shall parse uploaded resumes into structured Markdown and extract key fields including skills, experience, education, and certifications using local LLM-assisted parsing.
FR-03	The system shall generate a 384-dimensional dense vector embedding for each uploaded resume using the all-MiniLM-L6-v2 Sentence Transformer model.

ID	Requirement
FR-04	The system shall support automated scraping of job listings from LinkedIn, Indeed, Naukri, RemoteOK, and We Work Remotely via a Celery-Beat-scheduled Playwright pipeline.
FR-05	The system shall generate a 384-dimensional dense vector embedding for each scraped job listing and store it in PostgreSQL using the pgvector extension.
FR-06	The system shall use pgvector to retrieve a candidate set of job listings by cosine similarity and rank the candidates using semantic and keyword signals, returning a paginated list with configurable limit (1–100) and skip values.
FR-07	The system shall support filtering of matched jobs by workplace type (Remote, Hybrid, Onsite, Negotiable) and keyword search across job titles, company names, and descriptions.
FR-08	The system shall provide an ATS Keyword Diagnostic that analyses the gap between a resume and a specific job description, reporting missing keywords, match percentage, and categorised recommendations.
FR-09	The system shall generate tailored resume bullet points and cover letters using a local LLM (unsloth/Llama-3.2-3B-Instruct) via a RAG pipeline grounded in the candidate's parsed resume data, with real-time streaming output.
FR-10	The system shall support AI-generated company research summaries for target employers.
FR-11	The system shall cache all AI-generated outputs in PostgreSQL to avoid redundant computation for identical inputs.
FR-12	The system shall provide a Kanban board for application tracking across five stages: Interested, Applied, Interviewing, Offered, and Rejected.
FR-13	The system shall support export of AI-generated content and ATS diagnostic reports to DOCX and PDF formats.
FR-14	The system shall support management of multiple resumes through a Resume Vault with active resume selection.
FR-15	The system shall provide a configuration interface to switch AI computation between CPU and CUDA modes and between Demo (fast, heuristic) and Full (LLM-powered) inference modes without application restart.

4.5.2 Non-Functional Requirements

The non-functional requirements governing the system's quality attributes are specified as follows:

Table 4.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Mean latency of the two-stage hybrid job-matching endpoint POST /jobs/matches shall be below 10 milliseconds on the tested 100,000-row job collection.
NFR-02	Privacy	No user data — including resumes, parsed content, embeddings, or AI-generated outputs — shall be transmitted to any external server at any stage of the application lifecycle. All AI inference shall be performed locally.
NFR-03	Availability	The system shall implement graceful degradation across all AI components. If the primary LLM fails to load, a smaller fallback model shall be attempted. If all LLM models fail, the system shall degrade to heuristic regex-based parsing without crashing.
NFR-04	Scalability	The containerised architecture shall support horizontal scaling of scraper workers via Docker Compose replica configuration.
NFR-05	Portability	The system shall be deployable on any platform supporting Docker, including Linux, macOS, and Windows (via WSL2), with no platform-specific code paths.
NFR-06	Security	Endpoints accepting user input shall validate requests server-side through Pydantic models, typed parameters or explicit checks. File uploads shall be restricted to allowed extensions and a maximum size of 10 MB. Path traversal risks shall be mitigated through filename sanitisation. Administrative endpoints shall require API key authentication.
NFR-07	Usability	The frontend interface shall provide a responsive, dark-themed design with intuitive navigation across all functional modules.
NFR-08	Maintainability	All SQL queries shall use parameterised bindings to prevent SQL injection. Database sessions shall be managed through FastAPI's dependency injection system with automatic cleanup.

4.5.3 Use Case Analysis

The main actors are the Candidate and the Celery-Beat Scheduler. The Candidate performs the user-facing actions, while the scheduler triggers periodic scraping. The System is the application being modelled, rather than a separate external actor. The use cases are illustrated in Fig. 5.5.

Use Case 1, Upload and Parse Resume: The Candidate uploads a resume file. The System validates the file type and size, extracts text content, converts it to structured Markdown, parses key fields (skills, experience, education) using the local LLM,

generates a 384-dimensional embedding, and persists all data to PostgreSQL. The parsed resume is added to the Resume Vault.

Use Case 2, Discover Matching Jobs: The Candidate activates a resume from the Vault and initiates a job search. The System uses pgvector's HNSW index to retrieve a candidate set by cosine distance, then ranks those candidates using semantic and keyword signals. Results are returned as a paginated list with optional workplace-type and keyword filters.

Use Case 3, Run ATS Diagnostic: The Candidate selects a specific job from the matched results or provides a custom job description. The System performs a keyword gap analysis, computing match percentages for hard skills, soft skills, and industry terminology, and returns categorised recommendations for resume improvement.

Use Case 4, Generate Tailored Content: The Candidate requests a tailored resume or cover letter for a specific job. The System constructs a RAG prompt by retrieving the candidate's parsed resume data as context, feeds the prompt to the local LLM, and streams the generated content to the frontend in real time. The output is cached for future retrieval.

Use Case 5, Track Applications: The Candidate moves a job through the Kanban pipeline stages (Interested → Applied → Interviewing → Offered / Rejected), optionally attaching notes at each transition.

Use Case 6, Automated Job Scraping: The Celery-Beat Scheduler triggers periodic scraping tasks according to a configured schedule. The scraper worker launches Playwright browser instances, navigates job portal pages, extracts listing data, generates embeddings, and persists new jobs to PostgreSQL while filtering out duplicates via URL-based deduplication.

Chapter 5 - System Design

5.1 System Architecture and Overall Workflow

Fig. 5.1 shows the overall architecture of Job Scout: job boards and the candidate's resume are the inputs, the FastAPI backend coordinates the AI models, the Celery scrapers, Redis and PostgreSQL, and the outputs are tailored cover letters and application tracking. Fig. 5.2 presents the same system at the level of Docker services.

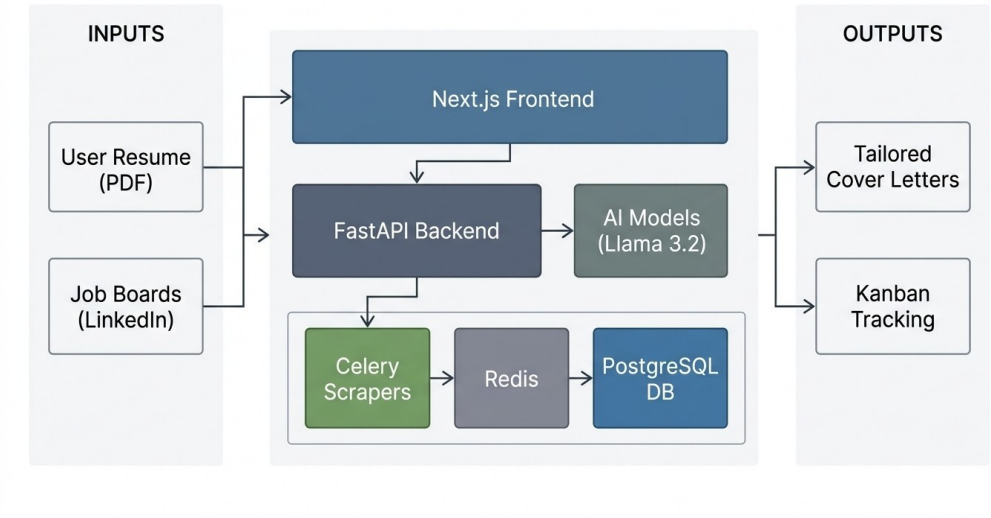


Fig. 5.1 Overall System Architecture Diagram

Docker Compose runs six services on a private application network. The FastAPI backend handles API requests, while the separate Celery worker performs scraping tasks. Redis carries queued tasks and supports caching, and PostgreSQL stores application data and embeddings. This separation lets longer-running background tasks run outside the request-handling path.

- **PostgreSQL Database Service (db):** hosts the relational data store with pgvector enabled. It stores job listings, resume data, user profiles, application-tracking records, embeddings and AI-generation cache entries. PostgreSQL is exposed internally on port 5432 and on host port 5435 for development. The pg_isready health check helps ensure dependent services start after the database is ready.
- **Redis Message Broker Service (redis):** provides message brokering for Celery task dispatch and result storage, and serves as the cache backend for FastAPI

responses. Redis is exposed internally on port 6379 and on host port 6380. The redis-cli ping health check is used to check service readiness.

- **FastAPI Backend Service (backend):** serves the REST API on port 8001 and handles resume parsing, embedding generation, matching, ATS diagnostics, local AI content generation, application tracking and document export. It waits for the database and Redis health checks, initialises the database schema through init_db.py, and runs under Uvicorn with hot reload enabled for development.
- **Celery Scraper Worker Service (scraper):** executes scraping tasks dispatched through the Redis broker. It launches Playwright browser instances to navigate job portals, extract listing data, infer workplace types, generate embeddings and save new jobs using URL-based deduplication. It depends on the database and Redis.
- **Celery-Beat Scheduler Service (celery-beat):** triggers periodic scraping tasks at configured intervals and enqueues them through Redis for the scraper worker. It does not execute the scraping tasks itself. It depends on Redis and the scraper service.
- **Next.js Frontend Service (frontend):** serves the client-facing web application on port 3000. Built with React 19 and Next.js 16, it provides job discovery, the Resume Vault, ATS diagnostics, the AI Copilot, the Kanban board, Market Radar and configuration controls. It communicates with the backend through REST API calls.

The six services expose PostgreSQL with pgvector on port 5435, Redis on port 6380, the FastAPI backend on port 8001 and the Next.js frontend on port 3000. Celery-Beat schedules scraping every 10 minutes by default. The backend and scraper use HF_HUB_OFFLINE=1 and TRANSFORMERS_OFFLINE=1 so Hugging Face model files are not downloaded during inference. The model files must be available locally before offline operation; the offline test in Chapter 7 confirmed that the tested parsing, embedding, matching and generation steps completed with external network access blocked.

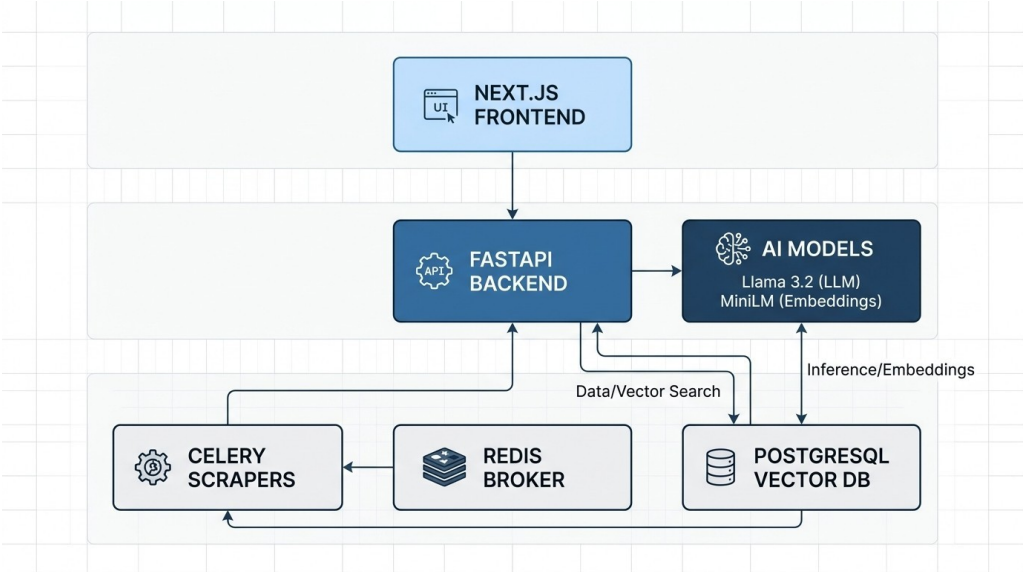


Fig. 5.2 Service-Level Architecture of the Dockerised Deployment

The overall workflow follows the Retrieval-Augmented Generation pattern shown in Fig. 5.3. In the ingestion stage, resumes are parsed and job descriptions are scraped; both are converted into embeddings by the all-MiniLM-L6-v2 model. In the storage and retrieval stage, the vectors are held in PostgreSQL with pgvector, and a hybrid query returns the top matching jobs for the active resume. In the generation stage, the matched job and the candidate's resume data are supplied as context to the local Llama-3.2-3B-Instruct model, which streams the tailored output back to the user.

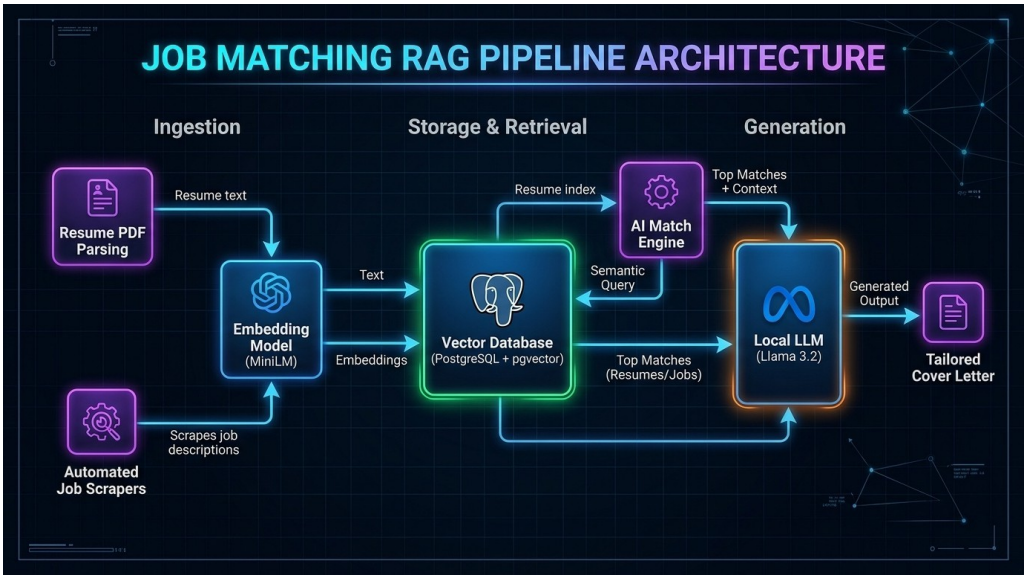


Fig. 5.3 Hybrid RAG Pipeline Workflow

5.2 Context Diagram

Fig. 5.4 shows the Level 0 context diagram. It represents two external entities: the Candidate, who uploads resumes, initiates searches, requests generated content and manages applications; and external job portals, which supply listings through the scraping pipeline. The Candidate exchanges inputs and outputs with Job Scout, while the portals supply job data to the system.

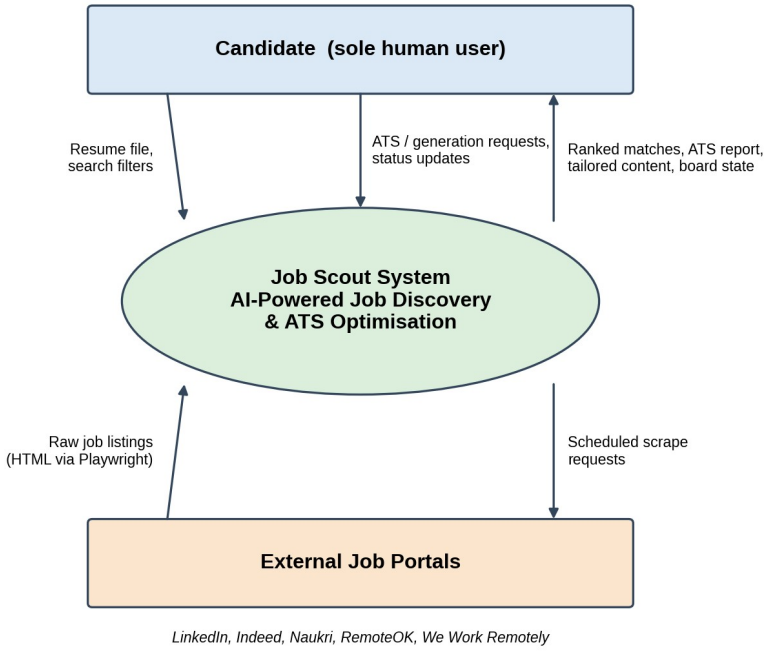


Fig. 5.4 Context Diagram (Level 0)

5.3 Use Case Diagram

Fig. 5.5 depicts the two actors and six use cases of the system. The Candidate performs UC1 to UC5, while the Celery-Beat Scheduler triggers UC6 autonomously. UC2 includes the selection of an active resume from the Resume Vault, and UC4 can be started directly from the result of UC3 through the Tailor Resume in Copilot action. Each use case is described in Section 4.5.3.

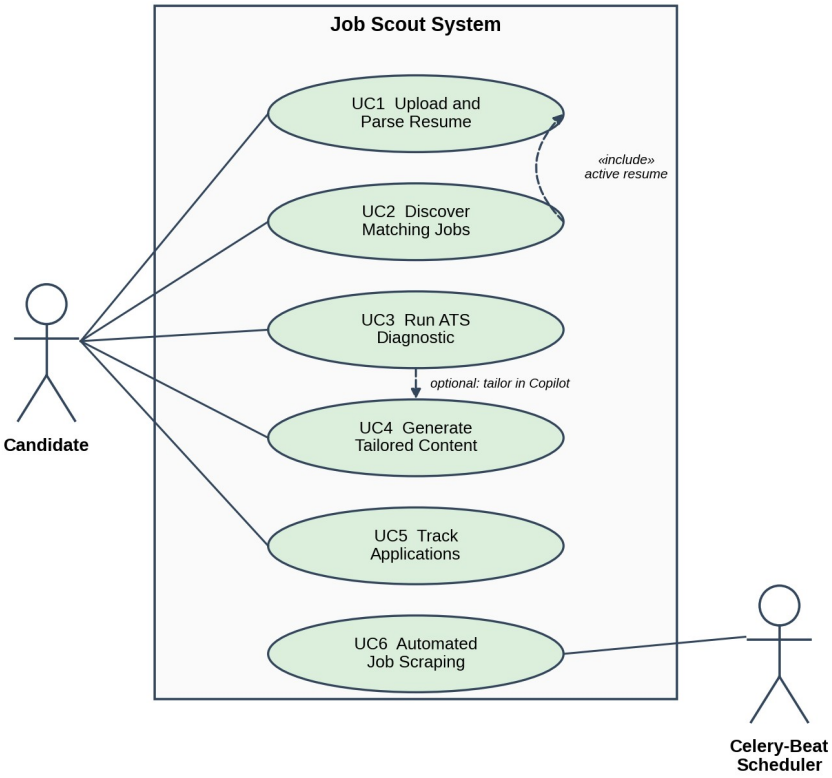


Fig. 5.5 Use Case Diagram of Job Scout

5.4 Activity Diagram

Fig. 5.6 shows the end-to-end activity flow across the Candidate and the system. The flow begins with resume upload and validation, continues through parsing and embedding, then proceeds to hybrid matching, ATS diagnosis, retrieval-augmented generation with caching and application-status updates on the Kanban board. Unsupported file types are rejected with HTTP 400, and uploads larger than 10 MB are rejected with HTTP 413 before parsing.

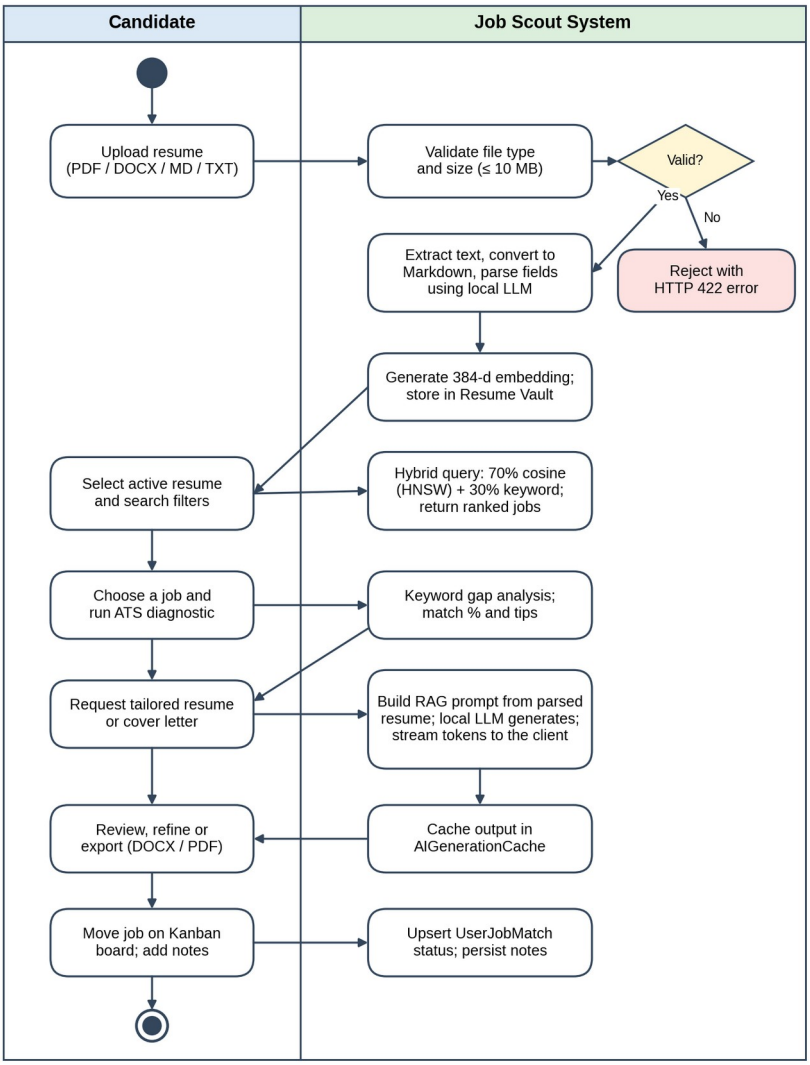


Fig. 5.6 Activity Diagram of the Candidate Workflow

5.5 Data Flow and Entity-Relationship Diagrams

5.5.1 Data Flow Diagram

Fig. 5.7 shows the Level 1 data flow diagram, which decomposes the system into the five major processes described below.

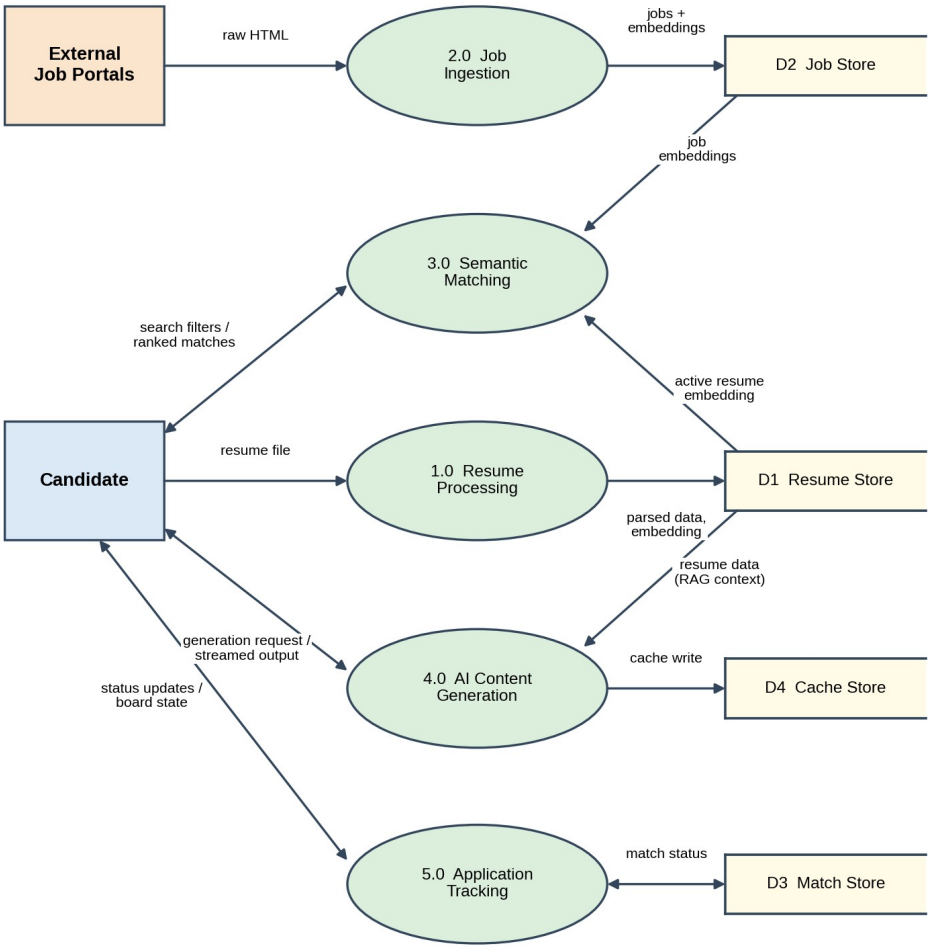


Fig. 5.7 Data Flow Diagram (Level 1)

The system decomposes into five major processes at Level 1:

Process 1.0, Resume Processing: receives a resume file from the Candidate, extracts its text, converts the content to Markdown, uses the local LLM to parse key fields and generates an embedding. The structured resume data and embedding are stored in the Resume Data Store.

Process 2.0, Job Ingestion: is triggered by Celery-Beat. Playwright opens job portal pages, extracts listing details, generates embeddings and stores new records in the Job Data Store. URL-based deduplication is used to skip records that are already present.

Process 3.0, Job Matching: retrieves the active resume embedding and job embeddings, calculates the hybrid score using semantic similarity (70%) and keyword relevance (30%), and returns a ranked list through the frontend.

Process 4.0, AI Content Generation: receives a request from the Candidate, retrieves parsed resume information as context, builds a prompt and invokes the local LLM. The response is streamed to the frontend, and completed outputs are stored in the AI generation cache.

Process 5.0, Application Tracking: receives status updates from the Candidate and stores application state in the UserJobMatch table. The Kanban interface retrieves the current state for display.

5.5.2 Entity-Relationship Diagram

Fig. 5.8 illustrates the intended logical relationships among the five core entities. In the current ORM implementation, the Resume model has no user_id column, and UserJobMatch stores user_id and job_id as indexed integers without declared foreign-key constraints. The relationships in the diagram should therefore be read as conceptual rather than database-enforced.

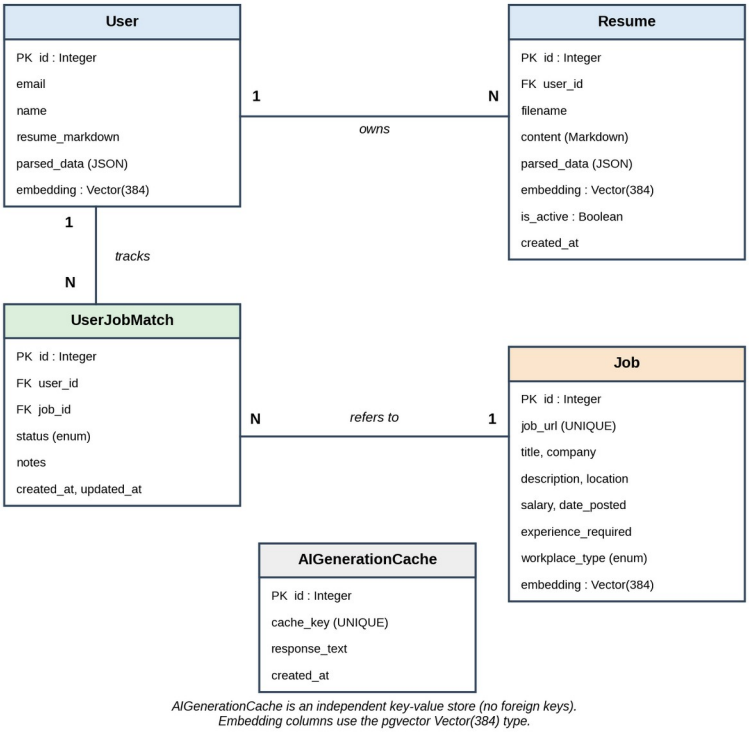


Fig. 5.8 Conceptual Entity-Relationship Diagram

5.6 Database Design

5.6.1 Entity-Relationship Description

The database schema comprises five core entities implemented as SQLAlchemy ORM models mapped to PostgreSQL tables:

- User (users table): stores the profile fields name and email, together with resume_markdown, parsed_data (JSON), a 384-dimensional embedding vector and a creation timestamp. The current deployment follows a single-user workflow, and the ORM model does not declare relationships to the other tables.
- Resume (resumes table): stores each resume's filename, Markdown text, parsed fields (JSON), 384-dimensional embedding, is_active flag and creation timestamp. The active-resume selection is managed by application logic; the current model does not include a user_id column.
- Job (jobs table): stores scraped job listings with title, company, description, location, salary, job_url (unique for deduplication), parsed metadata (JSON), a 384-dimensional embedding, experience_required (integer), workplace_type (string; expected values include remote, hybrid, onsite, negotiable and unspecified), date_posted and a creation timestamp. The title and company fields are indexed.
- UserJobMatch (user_job_matches table): records an application-tracking entry using indexed user_id and job_id integer fields, along with status, notes, created_at and updated_at. The current ORM model does not declare foreign-key constraints, so these identifiers are used at the application level.
- **AIGenerationCache (ai_generation_caches table):** Stores cached AI generation outputs with a unique cache_key (indexed for fast lookup) and the generated response_text. This entity implements the memoisation layer that prevents redundant LLM computation for identical input combinations.

The fields implemented in the current ORM models are summarised in Table 5.1. Fig. 5.8 provides a conceptual view of the intended relationships; some relationships shown there are not enforced by the current schema.

Table 5.1 Database Schema Definitions

Entity	Primary key	Core attributes	Relationships
User	id (Integer)	name, email, resume_markdown (Text), parsed_data (JSON), embedding (Vector 384), created_at	Profile record; no ORM relationship declarations
Resume	id (Integer)	filename, resume_markdown (Text), parsed_data (JSON), embedding (Vector 384), is_active, created_at	No user_id column in the current model
Job	id (Integer)	job_url (Unique), title, company, description, location, salary, parsed_data (JSON), embedding (Vector 384), experience_required, workplace_type, date_posted, created_at	Referenced by UserJobMatch.job_id in application logic
UserJobMatch	id (Integer)	user_id, job_id, status, notes, created_at, updated_at	user_id and job_id are indexed integers; no ORM foreign-key constraints
AIGenerationCache	id (Integer)	cache_key (Unique), response_text, created_at	Independent key-value store

5.6.2 Vector Storage and Indexing

The embedding columns in the User, Resume, and Job tables are defined using pgvector's Vector(384) type, which stores 384-dimensional floating-point vectors natively in PostgreSQL. To enable low-latency approximate nearest neighbour queries, an HNSW (Hierarchical Navigable Small World) index is created on the jobs.embedding column using the vector_cosine_ops operator class. This index pre-computes a multi-layered proximity graph over the job embedding vectors, enabling the cosine distance operator (<=>) to execute approximate similarity searches in O(log n) time rather than the O(n) time required by sequential scan. Benchmark measurements (Section 7.5.1) recorded, for 100,000 job rows, a mean query latency of 2.88 ms and a Recall@10 of

0.9565 with the query-time setting ef_search = 400, against 41.33 ms for an exact sequential scan; the recall depends strongly on ef_search (0.5800 at the default of 40).

The index is created with m = 16 and ef_construction = 128 (backend/init_db.py and backend/migrations/add_vector_index.sql); at query time, the pgvector parameter hnsw.ef_search sets the trade-off between recall and latency (Section 7.5.1).

5.6.3 Supporting Database Indexes

In addition to the HNSW vector index, the following supporting indexes are deployed to optimise query performance across the application:

- **idx_resume_is_active:** A partial index on resumes(is_active) filtered to WHERE is_active = true, optimising the frequent query for the currently active resume.
- **idx_jobs_workplace_type_lower:** A functional index on LOWER(workplace_type) in the jobs table, optimising case-insensitive workplace type filtering in the matching query.
- **idx_user_job_matches_composite:** A composite index on user_job_matches(user_id, job_id), optimising the LEFT JOIN in the matching query and the upsert operation in the application tracking endpoint.

5.7 User Interface Design

The user interface is a web application built with Next.js and React. It provides views for job discovery, the Resume Vault, the ATS diagnostic, the AI Copilot, the Kanban board, the Market Radar and the settings. Fig. 5.9, Fig. 5.10 and Fig. 5.11 are schematic wireframes of three of the most important screens, drawn to show the layout and the actions available on each; screenshots of the implemented interface are given in Appendix 2.

Job Discovery. This view lists ranked job matches with their match score, a workplace-type filter and a keyword search box. Each job card offers actions to run the ATS diagnostic, open the job in the AI Copilot, and mark the job as Interested.

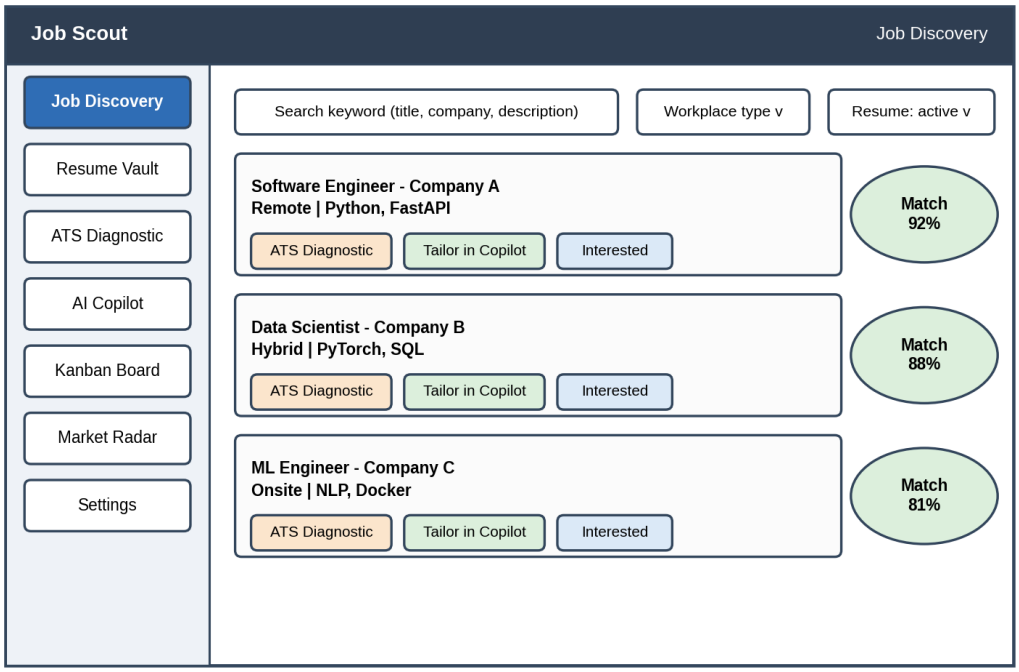


Fig. 5.9 UI Wireframe of the Job Discovery View

AI Copilot. The Copilot view offers four modes, tailored resume, cover letter, refinement, and company research. The target job description is shown on the left and the generated content streams into the right-hand pane token by token, with an option to export the result as DOCX or PDF.

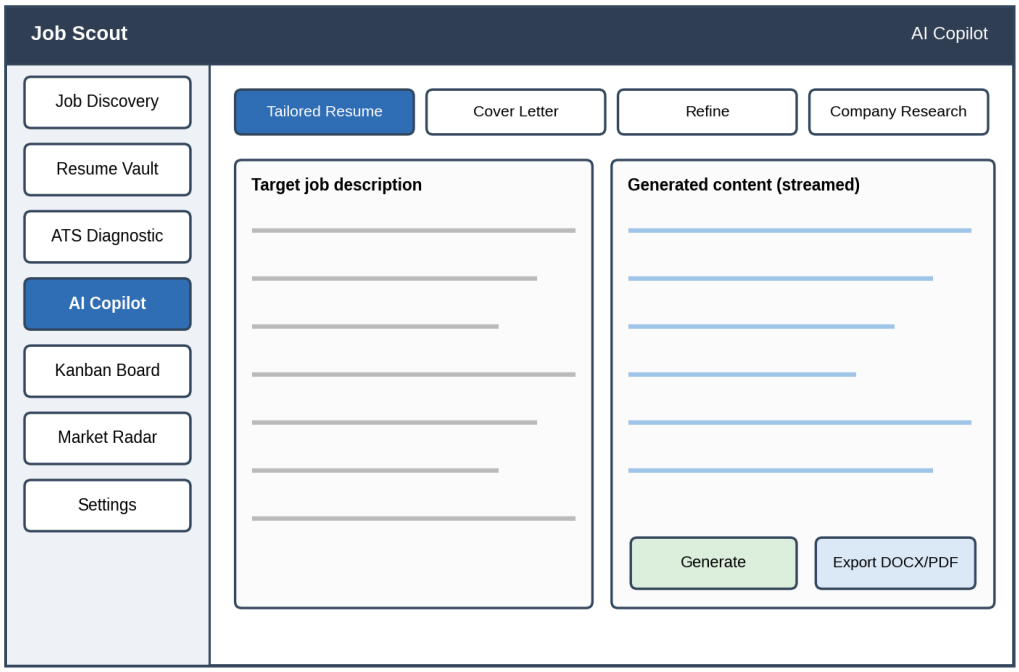


Fig. 5.10 UI Wireframe of the AI Copilot View

Kanban Board. Applications are tracked across five columns, Interested, Applied, Interviewing, Offered, and Rejected. Cards are moved between columns by drag and drop, and each card carries notes; status changes are persisted to the database.

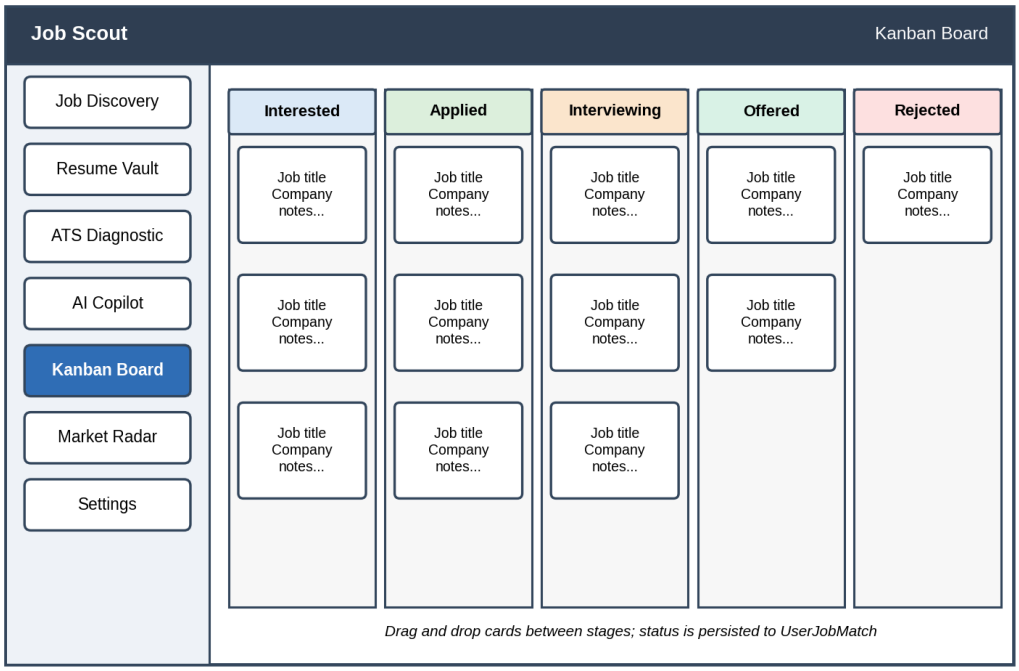


Fig. 5.11 UI Wireframe of the Kanban Board View

Other views. The Resume Vault lists all stored resumes with upload, activate, delete and export actions. The ATS Diagnostic is presented as a modal with matched keywords, missing keywords, an ATS compatibility percentage, and recommendations. The Market Radar shows aggregate statistics on job distribution by company, location, workplace type and posting recency, and the Settings view switches the computation device and inference mode.

5.8 Module Design

The system is decomposed into the following functional modules, each corresponding to a discrete frontend view and one or more backend service endpoints:

5.8.1 Resume Management Module (Resume Vault)

This module manages the candidate's stored resumes. The frontend component (VaultView.tsx) supports upload, activation, deletion and export. The backend exposes seven related endpoints: POST /resume/parse for upload and parsing; GET /resume/active and GET /resumes for retrieving the active resume and listing stored resumes; POST

/resumes/{resume_id}/activate for selecting an active resume; DELETE /resumes/{resume_id} and DELETE /resume/reset for deleting one resume or resetting stored resume data; and GET /resumes/{resume_id}/export for exporting a stored resume as DOCX or PDF. The upload pipeline validates the file, extracts text with MarkItDown and uses PyMuPDF for PDF fallback extraction or python-docx for DOC and DOCX fallback extraction. It then converts the content to Markdown, extracts structured fields with the local LLM, generates a 384-dimensional embedding with all-MiniLM-L6-v2 and stores the resulting data in PostgreSQL.

5.8.2 Job Discovery and Matching Module

This module implements the hybrid job-retrieval pipeline. The frontend component (the main job-listing view) displays ranked matches with similarity scores, workplace-type filters and keyword search. The POST /jobs/matches endpoint accepts the active resume's 384-dimensional embedding vector, validated by Pydantic, along with optional filter parameters (limit, skip, workplace_types and search_keyword). The matching service constructs a SQL query that combines cosine similarity with keyword relevance: 70% semantic score and 30% keyword score. It uses PostgreSQL regex matching for alphanumeric skills and LIKE matching for skills containing special characters (for example, C++, C# and .NET). Results are sorted by the combined score and joined with user_job_matches to include application status.

5.8.3 ATS Keyword Diagnostic Module

This module compares keyword sets from a candidate's resume and a target job description. The frontend component (ATSScoreModal.tsx) displays matched and missing keywords, a keyword-overlap percentage and recommendations. The POST /ai/ats-analyze endpoint extracts keyword sets from both documents, computes overlap metrics and returns a structured diagnostic report. A Tailor Resume in Copilot action passes the selected job context and parsed resume data to the generation interface. The diagnostic is deterministic: it uses token and n-gram matching and does not call the language model.

5.8.4 AI Copilot Module (Content Generation)

This module provides the interface for AI-assisted content generation. The frontend component (AITailorView.tsx) supports four modes: tailored resume generation, cover letter generation, content refinement and company research. The backend provides chunked responses for POST /ai/generate, POST /ai/refine and POST /ai/research. The separate POST /ai/cache/clear endpoint invalidates the generation cache. For resume and cover-letter requests, parsed resume information is supplied as context and the target job description is included where applicable. The prompt is passed to the local unsloth/Llama-3.2-3B-Instruct model through Hugging Face Transformers. FastAPI returns generated text as a chunked HTTP response, which the frontend displays incrementally. Completed generations are cached in the AIGenerationCache table using a key derived from request inputs.

5.8.5 Application Tracking Module (Kanban Board)

This module provides a visual pipeline for tracking job applications across five lifecycle stages: Interested, Applied, Interviewing, Offered, and Rejected. The frontend component (KanbanBoard.tsx) renders a drag-and-drop Kanban interface with per-card notes and status indicators. The backend endpoints POST /jobs/interest handles status transitions by upserting records in the UserJobMatch table, and GET /jobs/board retrieves all tracked jobs grouped by status with their associated job details via a JOIN query.

5.8.6 Automated Scraping Module

This module manages the scheduled collection of job listings from external portals.

5.8.7 Market Radar Module

This module (RadarView.tsx) provides analytical visualisations of the scraped job market data, presenting aggregate statistics on job distribution by company, location, workplace type, and posting recency. It consumes the same job data stored in PostgreSQL but presents it through an analytical lens, enabling the candidate to identify market trends and emerging opportunity clusters.

5.8.8 Configuration Module

This module allows users to change AI processing settings without restarting the application. The settings interface calls GET /ai/config and POST /ai/config to read or update the configuration. Options include the computation device (CPU or CUDA), inference mode (Demo for heuristic-based generation or Full for LLM-powered generation) and model selection. A device change is applied when the model is next loaded for generation.

5.9 API Design

The API follows standard HTTP method semantics. JSON is used for most requests and responses; the AI generation, refinement and research routes return chunked plain-text streams. Pydantic models and typed parameters enforce server-side validation. FastAPI generates an OpenAPI (Swagger) specification available at /docs. The principal endpoint groups are:

- Resume Management: POST /resume/parse, GET /resume/active, GET /resumes, POST /resumes/{resume_id}/activate, DELETE /resumes/{resume_id}, GET /resumes/{resume_id}/export and DELETE /resume/reset.
- Job Matching: POST /jobs/matches (accepts the active resume's 384-dimensional embedding, limit, skip, workplace_types and search_keyword).
- Job Scraping: POST /jobs/scrape (accepts search query, location and source portals), GET /tasks/{task_id} (polls task progress) and POST /tasks/batch-status (polls several tasks).
- AI Generation and ATS Diagnostics: POST /ai/generate (streams tailored resume or cover-letter text), POST /ai/refine (streams refinement), POST /ai/research (streams company research), POST /ai/ats-analyze (returns the ATS keyword diagnostic), POST /ai/cache/clear and DELETE /ai/cache (invalidate cached AI responses).
- **Application Tracking:** POST /jobs/interest (upsert status), GET /jobs/board (retrieve Kanban state).

- Document Export: POST /ai/export, POST /export/document and POST /export/ats-report (export AI content and diagnostic reports as DOCX, PDF or text where supported).
- **Configuration:** GET /ai/config (read current AI settings), POST /ai/config (update device/mode).
- System Administration: POST /system/shutdown, POST /system/restart, GET /api/admin/stats, all protected by API key authentication via the X-Admin-Key header.

The FastAPI application registers 29 application routes plus four documentation routes (/docs, /docs/oauth2-redirect, /openapi.json and /redoc). The routes not covered by the groups above are GET / (root status) and GET /user (active profile and resume data). Table 5.2 lists the registered application routes. The job-matching response is cached in Redis for 60 seconds.

Table 5.2 Registered Application Endpoints

Method	Path	Purpose
GET	/	Root operational endpoint
POST	/resume/parse	Parse and store an uploaded resume
GET	/resume/active	Active resume and profile of the user
DELETE	/resume/reset	Clear all stored resume data
GET	/user	Active user profile and resume data
GET	/resumes	List the resumes in the vault
DELETE	/resumes/{resume_id}	Delete a resume from the vault
POST	/resumes/{resume_id}/activate	Make a resume the active one
GET	/resumes/{resume_id}/export	Export a stored resume as DOCX or PDF
POST	/jobs/matches	Top job matches for an embedding (cached in Redis for 60 s)
POST	/jobs/scrape	Trigger background scraping for one or more sources
POST	/jobs/interest	Mark a job as Interested on the Kanban board
GET	/jobs/board	All jobs with their application status
POST	/ai/ats-analyze	Deterministic ATS score and keyword gap matrix
POST	/ai/generate	Tailored resume or cover letter (streamed, cached)

Method	Path	Purpose
POST	/ai/refine	Refine generated content on user instruction
POST	/ai/research	Company research sheet (streamed)
POST	/ai/export	Export generated content as DOCX, PDF or TXT
GET / POST	/ai/config	Read or switch the execution device (CPU, GPU, demo mode)
POST / DELETE	/ai/cache/clear, /ai/cache	Clear the cached AI responses
POST	/export/ats-report	Export the ATS report as PDF, DOCX or text
POST	/export/document	Export a document as DOCX, PDF or TXT
GET	/tasks/{task_id}	State of a Celery background task
POST	/tasks/batch-status	State of several background tasks
GET	/api/admin/stats	Telemetry for the admin dashboard
POST	/system/restart	Restart the Job Scout stack
POST	/system/shutdown	Shut the application down

5.10 Security Design

The security architecture of Job Scout is implemented across multiple layers:

- **Input Validation:** API endpoints validate incoming data through Pydantic models, typed parameters and explicit checks. The MatchRequest model checks that embeddings contain exactly 384 dimensions, limits range from 1 to 100, and skip values are non-negative. File uploads are restricted to .pdf, .docx, .doc, .md and .txt files up to 10 MB.
- **SQL Injection Prevention:** All database queries use parameterised bindings via SQLAlchemy's text() with named parameters. User-supplied values (skills, keywords, job descriptions) are never interpolated into SQL strings. This applies to the hybrid matching query's CASE/WHEN expressions, keyword search filters, and all CRUD operations.
- Path Traversal Mitigation: uploaded filenames are reduced to their base name with os.path.basename() and prefixed with a UUID. This reduces the risk of path traversal through a supplied filename.

- Administrative Endpoint Protection: shutdown, restart and admin-statistics endpoints check the X-Admin-Key header against ADMIN_API_KEY. If the variable is absent in the backend environment, these endpoints return HTTP 503; an incorrect key is rejected. The current Docker Compose configuration supplies a default value when ADMIN_API_KEY is omitted, so that value must be overridden before use beyond a local demonstration.
- **CORS Configuration:** Cross-Origin Resource Sharing is restricted to explicitly configured origins loaded from the CORS_ALLOWED_ORIGINS environment variable, defaulting to localhost ports 3000 and 3001 for development.
- Data Privacy by Design: the application runs AI inference locally and sets HF_HUB_OFFLINE=1 and TRANSFORMERS_OFFLINE=1 for the backend and scraper. These flags prevent Hugging Face model downloads during inference, but they do not replace general network controls. The offline test in Chapter 7 blocked external network egress and verified the defined resume parsing, embedding, matching and generation workflow.

Chapter 6 - System Implementation

This chapter describes how Job Scout is implemented, including its main algorithms and selected code excerpts. The backend uses Python tools including FastAPI, PyTorch, Hugging Face Transformers and Celery. The frontend uses TypeScript and Next.js, while PostgreSQL and pgvector provide relational and vector storage.

6.1 Module Descriptions

Table 6.1 maps each functional module to its frontend component, backend endpoints and implementation artefacts. The design of each module is described in Section 5.8.

Table 6.1 Modules and Their Implementation Artefacts

Module	Frontend component	Backend endpoints / services
Resume Vault	VaultView.tsx	POST /resume/parse; GET /resume/active; GET /resumes; POST /resumes/{id}/activate; DELETE /resumes/{id}; GET /resumes/{id}/export
Job Discovery and Matching	Main page job listing view	POST /jobs/matches; matching_service.py
ATS Keyword Diagnostic	ATSScoreModal.tsx	POST /ai/ats-analyze
AI Copilot	AITailorView.tsx	POST /ai/generate; POST /ai/refine; POST /ai/research; POST /ai/export; POST /ai/cache/clear; llm_service.py
Application Tracking	KanbanBoard.tsx	POST /jobs/interest; GET /jobs/board
Automated Scraping	—	POST /jobs/scrape; scraper/tasks.py; linkedin_scraper.py; indeed_scraper.py; naukri_scraper.py; remote_scrapers.py
Market Radar	RadarView.tsx	Reads scraped job data from PostgreSQL
Neural Profile	ProfileView.tsx	GET /user
Admin Dashboard and System Control	admin/page.tsx	GET /api/admin/stats; POST /system/shutdown; POST /system/restart
Configuration	Settings interface	GET /ai/config; POST /ai/config

6.2 Data Collection and Dataset Description

Job Scout does not train a model on a fixed public dataset. It uses the pre-trained all-MiniLM-L6-v2 embedding model and Llama-3.2-3B-Instruct for generation without project-specific fine-tuning. Data are collected at run time from scraped job listings and resumes uploaded by the candidate.

Job listings are collected by Playwright-based scrapers scheduled by Celery-Beat from the five portals in Table 6.2. For every listing the system stores the title, company, description, location, salary, job URL (a unique key used for deduplication), posting date, required years of experience, and an inferred workplace type (remote, hybrid, onsite, negotiable, or unspecified). Resumes may be uploaded in PDF, DOCX, DOC, MD or TXT format with a maximum size of 10 MB.

Table 6.2 Job Data Sources and Scrapers

Portal	Scraper module
LinkedIn	linkedin_scraper.py
Indeed	indeed_scraper.py
Naukri	naukri_scraper.py
RemoteOK and We Work Remotely	remote_scrapers.py

Two performance datasets were used. The HNSW vector-index sweep embedded real job descriptions and populated 10,000- and 100,000-row datasets by repeating those embeddings with small Gaussian perturbations. A separate API benchmark started from 2,000 distinct generated job descriptions before expanding to 10,000 and 100,000 rows. Both used 384-dimensional embeddings and were run on a laptop with a 12th Gen Intel Core i7-1255U processor (12 logical threads) and 15 GiB of RAM (Section 7.5). For retrieval-quality evaluation, 28 synthetic candidate profiles and 560 job listings across five categories were used, with 15,680 relevance labels assigned programmatically on a scale from 0 to 3 (Section 7.5.5).

6.3 Data Preprocessing

Resume preprocessing begins with text extraction according to the file type: Markdown and plain-text files are read directly, and other formats are converted to Markdown with MarkItDown, with PyMuPDF (PDF) and python-docx (DOCX) as

fallback extractors. The extracted text is structured as Markdown, and key fields (skills, experience, education and certifications) are then extracted with the local LLM. When the LLM is unavailable, heuristic regular-expression parsers extract years of experience and skills from the plain text.

Job preprocessing follows the three-phase pipeline of the scraping module: discovery of listing URLs, a batch deduplication query against existing job_url values, and enrichment, in which the detail page is opened, the full description is extracted, and the workplace type is inferred from textual signals before the record is stored.

6.4 Feature Representation and Extraction

Each resume and each job description is represented by a 384-dimensional dense embedding produced by the all-MiniLM-L6-v2 Sentence Transformer. In addition to these semantic features, the candidate's skill list is used to build keyword features: every skill is matched against the job title using a regular-expression word boundary for alphanumeric skills and a LIKE pattern for skills with special characters such as C++, C# and .NET. The ATS diagnostic additionally extracts keyword sets from the resume and the job description and computes their overlap.

6.5 Model Architecture and Algorithm Design

6.5.1 Embedding Model

All embeddings in Job Scout are produced by all-MiniLM-L6-v2, which generates 384-dimensional vectors [9]. The same model is used for resumes and job descriptions so that both are represented in the same vector space and can be compared using cosine similarity.

6.5.2 Hybrid Scoring Algorithm

Jobs are ranked by a composite score that combines semantic and keyword evidence:

```
match_score = 100 x ( 0.7 x (1 - cosine_distance(resume, job)) + 0.3 x
keyword_score )
keyword_score = (sum of per-skill weights) / (2.0 x number_of_skills)
```

For each skill, a match in the job title contributes a weight of 2.0, and a match found only in the description contributes 1.0. This normalisation keeps the keyword score between 0 and 1. The 70/30 semantic-to-keyword weighting was an engineering choice

set before the retrieval evaluation; Chapter 7 reports how it performed on the synthetic benchmark.

6.5.3 Generation Model and Fallback Strategy

Tailored content is generated by unsloth/Llama-3.2-3B-Instruct [39], running locally through the Hugging Face text-generation pipeline. The device is selected at runtime: the model uses float16 on CUDA and bfloat16 on CPU. If the primary model cannot be loaded, the service attempts to load TinyLlama-1.1B-Chat [40]. If both models are unavailable, resume-parsing paths that support heuristic extraction can use regular-expression-based fallbacks; this fallback does not provide equivalent LLM-generated content for every endpoint. Demo mode uses heuristic or template-based output, while Full mode uses the language model.

6.6 System and Model Implementation

6.6.1 Semantic Vector Indexing (pgvector)

To achieve low-latency semantic search, the system integrates the pgvector PostgreSQL extension. This allows the database to store high-dimensional floating-point arrays directly alongside relational data. The following SQLAlchemy ORM mapping demonstrates the schema definition for the Job entity:

```
class Job(Base):
    __tablename__ = "jobs"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String, index=True)
    company = Column(String, index=True)
    description = Column(Text)
    location = Column(String)
    salary = Column(String)
    job_url = Column(String, unique=True)
    parsed_data = Column(JSON)
    embedding = Column(Vector(384))  # 384 dimensions for local MiniLM embeddings
    experience_required = Column(Integer, default=0)
    workplace_type = Column(String, default="unspecified")
    date_posted = Column(String)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
```

An HNSW (Hierarchical Navigable Small World) index is applied to the embedding column to support approximate nearest-neighbour search. It reduces query time in the tested configurations, with a trade-off between latency and recall described in Chapter 7.

6.6.2 Local Large Language Model (LLM) Integration

Job Scout uses a locally hosted language model for AI-assisted writing. The model-loading code uses Hugging Face Transformers and attempts to load unsloth/Llama-3.2-3B-Instruct first, then TinyLlama-1.1B-Chat if the primary model cannot be loaded. Local inference reduces the need to send resume content to a hosted AI service; the accuracy of its outputs is evaluated separately in Chapter 7.

```
def get_local_llm(device=None):
    ...
    # CPU optimization: limit threads to avoid heavy context switching overhead
    if device == "cpu":
        cpu_cores = multiprocessing.cpu_count()
        optimal_threads = max(1, min(4, cpu_cores // 2))
        torch.set_num_threads(optimal_threads)

    # Configure loading precision
    model_kwargs = {"cache_dir": MODEL_CACHE, "low_cpu_mem_usage": True}
    if device == "cuda":
        model_kwargs["torch_dtype"] = torch.float16
    else:
        model_kwargs["torch_dtype"] = torch.bfloat16

    try:
        local_llm = pipeline(
            "text-generation",
            model=PARSER_LLM_MODEL_NAME,
            device=0 if device == "cuda" else -1,
            model_kwargs=model_kwargs,
        )
    except Exception as e:
        print(f"Attempting fallback to TinyLlama/TinyLlama-1.1B-Chat-v1.0 on
{device}...")
        local_llm = pipeline(
            "text-generation",
            model="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
            device=0 if device == "cuda" else -1,
            model_kwargs=fallback_kwargs,
        )
```

This engine generates tailored resume bullet points and cover letters using the resume and job information supplied as context for the request.

6.6.3 Hybrid RAG Matching Query

The matching service uses a hybrid query that combines semantic similarity with keyword evidence.

```
for i, skill in enumerate(skills):
    param_name = f"skill_{i}"
    # Alphanumeric skills use a precise word-boundary regex
    if re.match(r'^[a-zA-Z0-9\s\-]+$', skill):
        regex_pattern = r'\y' + re.escape(skill).replace(r'\ ', r'\s+') + r'\y'
        params[param_name] = regex_pattern
        case_whens.append(f"""
            (CASE WHEN j.title ~* :{param_name} THEN 2.0
                  WHEN j.description ~* :{param_name} THEN 1.0
```

```
                  ELSE 0.0 END)""")
    else:
        # Special-character skills (e.g. "C++", "C#", ".Net") use substring
search
        params[param_name] = f"%{skill}%"
        case_whens.append(f"""
            (CASE WHEN LOWER(j.title) LIKE :{param_name} THEN 2.0
                  WHEN LOWER(j.description) LIKE :{param_name} THEN 1.0
                  ELSE 0.0 END)""")

# Normalise: division by the maximum possible points (2.0 per skill)
keyword_score_expr = f"({' + '.join(case_whens)}) / {float(2.0 * len(skills))}"
# Composite: 70% semantic vector score + 30% keyword score
score_calculation = (f"(0.7 * (1 - (j.embedding <=> :embedding)) "
                     f"+ 0.3 * ({keyword_score_expr})) * 100")
```

The query binds skill values as parameters instead of inserting user-supplied skill strings directly into SQL. This helps prevent SQL injection through these inputs; the security tests for the matching query are summarised in Chapter 7.

Index use in the hybrid query. EXPLAIN (ANALYZE) of the original single-stage query showed that PostgreSQL does not use the HNSW index. The ORDER BY clause sorts by a computed expression that combines the vector distance with the keyword score, so the planner cannot use the index to order the rows and falls back to a sequential scan with a top-N sort (130.6 ms at 100,000 rows, Section 7.5.1). The matching service therefore implements a two-stage query, which is the default (the environment variable HYBRID_MODE=legacy selects the single-stage query). The first stage retrieves a candidate set with a pure approximate-nearest-neighbour query that can use the index, and the second stage computes the keyword score and the composite rank only for those candidates; the number of candidates is the larger of 100 and five times the number of requested rows (candidate_k = max(100, (limit + skip) x 5)). The core of the query is:

```
WITH candidates AS (
    SELECT j.id, j.title, j.company, j.description, j.embedding, ...
    FROM jobs j
    {filter_str}
    ORDER BY j.embedding <=> :embedding
    LIMIT :candidate_k
)
SELECT c.id, c.title, c.company, ...,
       (1 - (c.embedding <=> :embedding)) * 100 AS vector_score,
       ({cte_keyword_expr}) * 100 AS keyword_score,
       ({cte_score_calc}) AS match_score
FROM candidates c
LEFT JOIN user_job_matches m ON c.id = m.job_id
WHERE (m.job_id IS NULL OR m.status = 'rejected')
ORDER BY ({cte_order_expr}) DESC
LIMIT :limit OFFSET :skip
```

Because the second stage re-ranks only the candidates, the result can differ from the exact single-stage ranking: the mean overlap of the top-10 results was 90.0% at 10,000

rows and 68.2% at 100,000 rows (Section 7.5.1). The number of rows that a pgvector HNSW scan returns is limited by the query-time parameter hnsw.ef_search, so the service runs SET LOCAL hnsw.ef_search before the query, with the larger of the configured value (the environment variable HNSW_EF_SEARCH, default 400) and candidate_k.

6.6.4 Asynchronous Task Queueing (Celery)

Job listing aggregation operates asynchronously to prevent blocking the main API thread. The Celery worker processes job links, embeds them in batches to optimize CPU/GPU utilization, and inserts them into the database.

```
@app.task(bind=True)
def scheduled_scrape(self):
    # Automated task triggered by Celery Beat every 10 minutes
    return scrape_and_process_jobs.apply_async(
        args=["Software Engineer", "Remote", 15, "linkedin"])


@app.task(bind=True)
def scrape_and_process_jobs(self, keyword, location, limit=10,
source="linkedin"):
    db: Session = SessionLocal()
    try:
        ...
        # Filter duplicates across all sources
        all_urls = [j.get('job_url') for j in jobs_data if j.get('job_url')]
        existing_urls = set(
            row[0] for row in
db.query(Job.job_url).filter(Job.job_url.in_(all_urls)).all()
        ) if all_urls else set()
        new_jobs = [j for j in jobs_data
                    if j.get('job_url') and j['job_url'] not in existing_urls]

        for job_info in new_jobs:
            content = job_info.get('description') or f"{job_info['title']} at
{job_info['company']}"
            embedding = generate_embedding(content)
            db.add(Job(title=job_info.get('title'), job_url=job_info['job_url'],
                       embedding=embedding, ...))
        db.commit()
    finally:
        db.close()
```

Batch URL deduplication checks the candidate URLs in a single database query before inserting new records. This avoids issuing a separate existence query for every listing.

6.6.5 Frontend Streaming Processing (Next.js)

The backend returns generated text as a chunked HTTP response. The Next.js frontend reads the chunks and updates the interface as new text arrives.

```
const response = await fetch(`${apiHost}/ai/generate`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(payload)
});

const reader = response.body?.getReader();
const decoder = new TextDecoder();
let done = false;
let accumulated = '';

while (!done) {
  const { value, done: doneReading } = await reader.read();
  done = doneReading;
  if (value) {
    const chunk = decoder.decode(value, { stream: !done });
    accumulated += chunk;
    setAiResult(accumulated);
  }
}
```

The stream reader decodes each response chunk and appends it to the displayed result, so the interface shows the generated text as it arrives.

6.7 Validation Checks

Input validation is enforced on the server so that malformed or malicious requests are rejected before they reach the database or the models. Table 6.3 summarises the checks implemented in Job Scout.

Table 6.3 Validation Checks Implemented in Job Scout

Check	Where applied	Behaviour on failure
Embedding length must be exactly 384	MatchRequest (Pydantic validator)	HTTP 422 Unprocessable Entity
Pagination limit between 1 and 100; skip not negative	MatchRequest (Pydantic validator)	HTTP 422 Unprocessable Entity
File extension in .pdf, .docx, .doc, .md, .txt and size up to 10 MB	Resume upload endpoint	HTTP 400 for an unsupported type; HTTP 413 above 10 MB
Filename sanitised with os.path.basename() and prefixed with a UUID	Resume upload endpoint	Sanitised base name and UUID reduce path-traversal risk
Admin API key supplied in the X-Admin-Key header	Shutdown, restart and admin statistics endpoints	HTTP 503 if the key is not configured; access denied otherwise
Parameterised SQL bindings for all user-supplied values	Matching query, keyword search, CRUD operations	Injection payloads treated as data

Check	Where applied	Behaviour on failure
Unique constraint on job_url and batch deduplication	Job table and scraping task	Duplicate listings skipped
CORS limited to configured origins	FastAPI middleware	Cross-origin requests from other origins blocked
Model-load fallback chain	LLM service	Attempts TinyLlama as fallback; heuristic extraction is used where supported if both models fail

6.8 User Interface Implementation

The frontend is implemented in TypeScript with Next.js 16, React 19 and Tailwind CSS 4. Each view of the interface corresponds to one component: VaultView.tsx (Resume Vault), ATSScoreModal.tsx (ATS diagnostic), AITailorView.tsx (AI Copilot), KanbanBoard.tsx (application tracking) and RadarView.tsx (Market Radar). The components communicate with the backend only through REST calls, and the AI Copilot consumes the streamed response as shown in Section 6.6.5. The wireframes of the main screens are given in Section 5.7 and screenshots of the implemented interface are given in Appendix 2.

Chapter 7 - Testing and Evaluation

7.1 Test Cases

Testing assessed selected functional, integration, security and performance requirements of Job Scout. Unit tests checked isolated logic and validation; integration tests exercised components together; system and security tests covered selected end-to-end behaviour; and benchmark scripts measured retrieval and generation performance.

The automated test suite in tests/ was run with pytest. It collected 126 tests, all of which passed in 14.60 seconds. The suite is included in the continuous-integration workflow at .github/workflows/tests.yml. Overall backend line coverage was 79%; Table 7.2 provides the module-level figures.

Table 7.1 Test Strategy

Level	Scope	Method
Unit testing	Pydantic validators, heuristic parser, scoring maths, matching service, LLM fallback and service logic, index definitions, ATS, export and resume services	pytest (tests/unit), 102 tests
Integration testing	ORM and connection pool, Celery scraping task, streaming and cache, REST routes	pytest (tests/integration), 20 tests
System and security testing	SQL injection, admin key, upload limits, CORS, Kanban persistence, offline operation	pytest (tests/security), 4 tests, and scripts/benchmarks/test_offline_operation.py
Performance testing	Vector search latency, HNSW parameter sweep, real API latency, LLM cache, CPU generation throughput, memory	Scripts in scripts/benchmarks with JSON outputs
Retrieval and generation evaluation	Precision, recall, MRR and NDCG of the ranking strategies; grounding of generated text	Scripts and labelled data in evaluation/

Table 7.2 Measured Backend Code Coverage

Module	Line coverage
Overall backend (1,792 statements)	79%

Module	Line coverage
app.main	72%
app.services.llm_service	79%
app.services.llm_fallback	66%
app.services.matching_service	75%
app.services.resume_service	84%
app.services.ats_service	88%
app.services.export_service	93%
app.models.models, app.core.database, app.core.config	100%

Coverage was lowest in the LLM fallback module (66%) and the API module (72%). Tests mock some model-related behaviour, so they do not exercise every path that loads and runs the real models.

7.2 Unit Testing

Unit testing focuses on verifying the smallest testable parts of the application in isolation. In Job Scout, unit tests were implemented for critical backend utility functions and data validators using the pytest framework.

Table 7.3 Unit Test Cases and Results

Test case ID	Module	Description	Expected output	Status
UTC-01	Database	Verify that idx_jobs_embedding_hnsw, idx_resume_is_active, idx_jobs_workplace_type_lower and idx_user_job_matches_composite exist in pg_indexes	All four indexes present	Pass
UTC-02	API validator	Reject MatchRequest with an embedding length other than 384	HTTP 422 Unprocessable Entity	Pass
UTC-03	API validator	Reject MatchRequest with limit above 100 or skip below 0	HTTP 422 Unprocessable Entity	Pass

Test case ID	Module	Description	Expected output	Status
UTC-04	Heuristic parser	Experience-regex variations (7 tests, for example “5+ yrs req” returns 5) and parsing of 50 synthetic resumes (50 tests)	Expected values returned without the LLM	Pass
UTC-05	LLM fallback	Failure of the primary model, then failure of both models (2 tests)	TinyLlama loaded; then heuristic path; no crash	Pass
UTC-06	Upload validation	Directory traversal in the uploaded filename	Basename plus UUID filename	Pass
UTC-07	Keyword maths	Word-boundary regex, substring match for special characters, title versus description weights, normalisation	Score within [0, 1]	Pass
UTC-08	Services	ATS service (5 tests), export service (4), resume parsing (7), LLM service logic (19), matching service (3)	Expected outputs from each function	Pass

7.3 Integration Testing

Integration tests exercise several components together against a real PostgreSQL instance with pgvector.

Table 7.4 Integration Test Cases and Results

Test case ID	Interface	Description	Expected output	Status
INT-01	API and PostgreSQL	ORM create, read, update and delete operations; connection pool behaviour	Operations succeed; no leaked sessions	Pass
INT-02	Celery task and database	Scraping task run against stub data that contains an existing job_url	Duplicates skipped; new rows stored with 384-dimensional embeddings	Pass
INT-03 and INT-04	Streaming and cache	Request to /ai/generate, then the identical request again (one test)	Chunked text/plain response; second response served from AIGenerationCache	Pass

Test case ID	Interface	Description	Expected output	Status
INT-05	REST routes	Root, job board and interest, resume CRUD and activation, AI configuration, document export, ATS analysis and report export, cache clearing, admin statistics (17 tests)	Expected status codes and payloads	Pass

7.4 System Testing

System-level behaviour was verified by the security suite and by the offline-operation audit, whose results are listed in Table 7.5.

Table 7.5 System and Security Test Cases and Results

Test case ID	Scenario	Expected output	Status
SEC-01	SQL-injection payloads in the skills and search keyword fields	Payloads treated as data; no injection	Pass
SEC-02	Admin endpoints without and with an invalid X-Admin-Key	HTTP 503 when ADMIN_API_KEY is not configured; 401 or 403 for an invalid key	Pass
SEC-03	Oversized upload, disallowed extension, request from a non-permitted origin	HTTP 413; HTTP 400; request blocked by CORS	Pass
SEC-04	Kanban status change followed by a new database session	Status persists across sessions	Pass
OFF-01	Parse, embed, match and generate with external network access blocked and HF_HUB_OFFLINE=1, TRANSFORMERS_OFFLINE=1	All stages succeed (see Section 7.5.4)	Pass

7.5 Model and Performance Evaluation

Performance measurements were collected using scripts in scripts/benchmarks and evaluation/. The raw JSON outputs are stored in the repository. The tests ran on a laptop with a 12th Gen Intel Core i7-1255U processor (12 logical threads), 15 GiB RAM, no

GPU, Linux (Fedora, kernel 7.2.8-200.fc44.x86_64) and Docker 29.8.1 on 11 October 2026.

7.5.1 Vector Search Latency and HNSW Parameters

The vector index was evaluated on 10,000 and 100,000 job rows of 384-dimensional embeddings. The rows were produced by embedding real job descriptions with all-MiniLM-L6-v2 and replicating them to the target size with small Gaussian noise (σ = 0.01) on the unit hypersphere; 200 queries were run per setting, and Recall@10 was measured against the exact result of a sequential scan. A grid of HNSW build parameters (m in {16, 32}, ef_construction in {64, 128, 200}) and of the query-time parameter ef_search in {40, 100, 200, 400} was swept; Table 7.7 and Table 7.8 give the complete results, and Table 7.6 the key configurations.

Table 7.6 Vector Search Latency in Milliseconds

Rows	Method	Mean	p50	p95	p99	Recall@10
10,000	Sequential scan (exact)	4.17	3.85	6.83	8.97	1.0000
10,000	HNSW, ef_search = 40	0.64	0.66	0.78	0.91	0.8795
10,000	HNSW, ef_search = 400	1.76	1.62	2.65	3.31	1.0000
100,000	Sequential scan (exact)	41.33	40.15	44.79	68.93	1.0000
100,000	HNSW, ef_search = 40	1.20	1.06	1.98	2.52	0.5800
100,000	HNSW, ef_search = 400	2.88	2.70	4.59	6.69	0.9565

The production index uses m = 16 and ef_construction = 128. With the query-time setting ef_search = 400, this index reached a Recall@10 of 0.9565 at 100,000 rows with a mean latency of 2.88 ms (p95 4.59 ms), below the 10 ms limit of NFR-01; at 10,000 rows the same setting gave a recall of 1.0000 at 1.76 ms against 4.17 ms for the sequential scan. Recall depends strongly on ef_search: at the pgvector default of 40 the same index reached a recall of only 0.8795 at 10,000 rows and 0.5800 at 100,000 rows, although its latency was lower (0.64 ms and 1.20 ms). Of the 24 parameter combinations, 15 reached a recall of at least 0.95 at 10,000 rows and only 4 at 100,000 rows; the combination used here (m = 16, ef_construction = 128, ef_search = 400) is the cheapest to build of the four. Raising m to 32 also reached the target but increased the build time at 100,000 rows from 104.99 s to 275.21 s. The application therefore sets hnsw.ef_search explicitly (Section 6.6.3).

Table 7.7 HNSW Parameter Sweep, 10,000 Rows

m	ef_constr.	ef_search	Build (s)	Recall@10	Mean	p50	p95
16	64	40	1.66	0.8985	0.48	0.47	0.59
16	64	100	1.66	0.9145	0.60	0.58	0.79
16	64	200	1.66	0.9220	0.85	0.74	1.42
16	64	400	1.66	0.9220	1.33	1.26	1.89
16	128	40	2.48	0.8795	0.64	0.66	0.78
16	128	100	2.48	0.9275	0.99	0.98	1.25
16	128	200	2.48	0.9400	1.05	0.97	1.52
16	128	400	2.48	1.0000	1.76	1.62	2.65
16	200	40	3.48	0.9750	0.68	0.67	0.90
16	200	100	3.48	0.9875	0.84	0.82	1.02
16	200	200	3.48	0.9950	0.98	0.95	1.31
16	200	400	3.48	1.0000	1.45	1.39	1.92
32	64	40	4.40	0.9475	0.71	0.60	1.28
32	64	100	4.40	0.9885	0.84	0.80	1.21
32	64	200	4.40	0.9910	1.55	1.43	2.47
32	64	400	4.40	0.9915	2.08	1.92	2.81
32	128	40	5.95	0.9910	0.53	0.51	0.63
32	128	100	5.95	0.9950	0.71	0.70	0.83
32	128	200	5.95	1.0000	1.23	1.09	1.78
32	128	400	5.95	1.0000	1.95	1.85	2.70
32	200	40	7.13	0.9465	0.72	0.72	0.90
32	200	100	7.13	0.9600	0.99	0.96	1.23
32	200	200	7.13	0.9950	1.29	1.24	1.72
32	200	400	7.13	1.0000	1.87	1.64	2.96

Table 7.8 HNSW Parameter Sweep, 100,000 Rows

m	ef_constr.	ef_search	Build (s)	Recall@10	Mean	p50	p95
16	64	40	88.73	0.4680	1.06	0.90	1.84
16	64	100	88.73	0.6325	1.21	1.08	2.16
16	64	200	88.73	0.8380	1.90	1.73	3.26
16	64	400	88.73	0.8825	2.66	2.51	4.16
16	128	40	104.99	0.5800	1.20	1.06	1.98

m	ef_constr.	ef_search	Build (s)	Recall@10	Mean	p50	p95
16	128	100	104.99	0.8255	1.45	1.32	2.38
16	128	200	104.99	0.9130	2.11	1.98	3.39
16	128	400	104.99	0.9565	2.88	2.70	4.59
16	200	40	120.55	0.6875	1.44	1.20	3.02
16	200	100	120.55	0.8530	1.65	1.52	2.76
16	200	200	120.55	0.9170	2.19	2.10	3.30
16	200	400	120.55	0.9260	2.89	2.77	4.55
32	64	40	249.68	0.5290	1.19	0.88	2.41
32	64	100	249.68	0.6900	1.41	1.28	2.30
32	64	200	249.68	0.8190	2.12	1.95	3.42
32	64	400	249.68	0.8560	3.49	3.37	5.42
32	128	40	275.21	0.7195	1.50	1.25	3.05
32	128	100	275.21	0.8945	1.96	1.86	3.05
32	128	200	275.21	0.9435	2.77	2.62	4.10
32	128	400	275.21	0.9820	3.69	3.49	5.41
32	200	40	310.98	0.8075	1.79	1.60	3.30
32	200	100	310.98	0.9285	2.08	2.02	3.30
32	200	200	310.98	0.9820	2.48	2.42	3.81
32	200	400	310.98	0.9835	3.28	3.14	4.87

The sequential-scan baseline at 100,000 rows was measured once during the parameter sweep at 41.33 ms and again across three runs at 29.31 ms, 29.79 ms and 28.68 ms (mean 29.26 ms). This variation should be kept in mind when comparing the scan with the HNSW query. Depending on which scan measurement is used, HNSW was about 10 to 14 times faster. The benchmark rows were replicated from a limited set of real descriptions, with small noise added to their embeddings, which created clusters of near-duplicate vectors. A separate test using 2,000 distinct generated job descriptions reached Recall@10 of 1.0000 at ef_search values of 40, 100 and 400, but that smaller dataset does not establish performance at 100,000 rows. An earlier run using uniformly random vectors had much lower recall and was not used as the main benchmark because those vectors did not reflect the structure of embeddings from job descriptions.

Hybrid query. The single-stage hybrid query, which sorts by the composite score, does not use the HNSW index. In the benchmark script it took 44.55 ms at 10,000 rows and 130.6 ms at 100,000 rows, against 1.35 ms and 2.45 ms for the two-stage query of Section 6.6.3 (33.1 and 53.3 times faster); EXPLAIN (ANALYZE) shows the two-stage plan sorting only the candidate rows. In that benchmark the top-10 results of the two queries overlapped by 90.0% at 10,000 rows (Kendall $\tau = 0.90$) and 68.2% at 100,000 rows ($\tau = 0.82$), with ef_search = 100. Table 7.9 gives the latencies.

Table 7.9 Single-Stage and Two-Stage Hybrid Query in the Benchmark Script

Rows	Query	Mean (ms)	p50 (ms)	p95 (ms)	Top-10 overlap	Kendall τ
10,000	Single-stage (legacy)	44.55	44.20	45.77	—	—
10,000	Two-stage	1.35	1.36	1.50	90.0%	0.90
100,000	Single-stage (legacy)	130.60	128.54	136.55	—	—
100,000	Two-stage	2.45	2.43	3.74	68.2%	0.82

The production route POST /jobs/matches was also evaluated through the API using 100 queries, including request validation and JSON serialisation (Table 7.10). The two-stage query averaged 8.43 ms at 10,000 rows and 9.08 ms at 100,000 rows, compared with 90.01 ms and 890.88 ms for the single-stage query. At 100,000 rows, the two-stage mean is below the 10 ms target, but its 95th and 99th percentiles were 11.13 ms and 12.05 ms. Recall values of 0.7310 and 0.7500 for the two-stage query were recorded in this benchmark; the benchmark file does not fully define this measure. Since the single-stage query also receives recall values below 1.0 despite scoring every row, this measure should not be interpreted as pure HNSW index recall.

Table 7.10 End-to-End Latency of POST /jobs/matches

Rows	Mode	Mean (ms)	p50	p95	p99	Recall@10
10,000	Two-stage	8.43	8.10	10.03	10.56	0.7310
10,000	Single-stage (legacy)	90.01	89.38	94.76	98.41	0.7300
100,000	Two-stage	9.08	8.74	11.13	12.05	0.7500
100,000	Single-stage (legacy)	890.88	890.62	912.03	924.41	0.6730

7.5.2 LLM Response Cache

The AIGenerationCache was evaluated in two modes with five paired repetitions each. In full CPU mode, with unsloth/Llama-3.2-3B-Instruct in bfloat16, 128 requested tokens and the CPU threads clamped to 4, the cold generation averaged 61.33 s (57.23 s to 66.61 s per repetition; 129 tokens generated from prompts of 101 to 104 tokens), and the identical request served from the SHA-256-keyed cache averaged 2.00 ms, a reduction by a factor of 30,637. In demo mode, in which a template generator streams about 90 words with a 30 ms pause per word, the cold request took 2.52 s and the cached one 2.44 ms (a factor of 1,030). The demo mode exists to demonstrate the interface without a GPU or 6 GB of memory; the full CPU figures are the ones that describe real generation.

7.5.3 CPU Generation Throughput and Memory

In the throughput benchmark, unsloth/Llama-3.2-3B-Instruct ran in bfloat16 on the CPU with 10 threads and generated 129 tokens in 70.79 s, a throughput of 1.82 tokens per second. In the cache benchmark, with the threads clamped to 4, the full-length generations ran at 2.05 to 2.25 tokens per second (mean 2.10, from 129 tokens in 61.33 s). The thread count therefore influences the speed, and on this machine the clamped setting was not slower. The memory footprint of the backend process was measured with its resident set size (RSS): 15.94 MB at baseline, 875.93 MB after the embedding model was loaded, and a peak of 7,225.99 MB (7.06 GB) during inference, a model overhead of 7,210.05 MB on a machine with 15 GiB of RAM. These figures are the basis of the hardware specification in Section 3.1. No GPU was available, so GPU generation speed was not measured.

7.5.4 Offline Operation

The offline audit set HF_HUB_OFFLINE=1 and TRANSFORMERS_OFFLINE=1 and blocked external network egress at the socket level. Resume parsing, embedding generation, hybrid matching (145.55 ms) and local text generation all completed. This result applies to the steps exercised by the audit after model files were available; it does not mean that initial setup or scheduled scraping can operate without internet access.

7.5.5 Retrieval Quality

Retrieval quality was evaluated on 28 synthetic candidate profiles and 560 job listings, giving 15,680 profile–job relevance labels graded from 0 to 3. The listings fall

into five categories: 240 canonical engineering postings, 80 paraphrased postings that describe the same work with no keyword overlap, 80 keyword-distractor postings (for example a recruiter or a sales role that mentions the same technologies), 80 adjacent technical roles, and 80 postings from unrelated fields. The labels were assigned programmatically from the role taxonomy of the dataset generator, independently of the keyword counts and vector distances used by the ranking code; they were not produced by human annotators.

An earlier evaluation using 12 profiles and 220 postings was discarded because its relevance labels were derived from overlap between candidate skills and listed job skills. Since the keyword-ranking method used the same information, that evaluation favoured keyword matching by construction.

Keyword-only and semantic-only ranking were scored in Python; the production hybrid ($\alpha = 0.7$, where α is the weight of the vector score) was scored through the real SQL queries, both the single-stage query and the two-stage query with candidate_k = 50, 100, 200 and 500. Table 7.11 reports precision at 5 and 10, recall at 10, mean reciprocal rank (MRR) and NDCG at 10, with bootstrap 95% confidence intervals (1,000 resamples).

Table 7.11 Retrieval Quality of the Ranking Strategies

Strategy	P@5	P@10	R@10	MRR	NDCG@10
Keyword-only (Python)	0.921 [0.829, 0.986]	0.889 [0.786, 0.975]	0.148 [0.125, 0.172]	0.976 [0.929, 1.000]	0.896 [0.804, 0.971]
Semantic-only (Python)	0.936 [0.850, 0.993]	0.932 [0.854, 0.986]	0.153 [0.132, 0.176]	0.945 [0.859, 1.000]	0.923 [0.836, 0.985]
Hybrid, single-stage SQL	0.986 [0.957, 1.000]	0.989 [0.968, 1.000]	0.162 [0.144, 0.182]	1.000 [1.000, 1.000]	0.985 [0.963, 1.000]
Hybrid, two-stage SQL, k = 50	0.986 [0.957, 1.000]	0.989 [0.957, 1.000]	0.162 [0.143, 0.183]	1.000 [1.000, 1.000]	0.985 [0.963, 1.000]
Hybrid, two-stage SQL, k = 100	0.986 [0.957, 1.000]	0.989 [0.968, 1.000]	0.162 [0.144, 0.183]	1.000 [1.000, 1.000]	0.985 [0.960, 1.000]
Hybrid, two-stage SQL, k = 200	0.986 [0.957, 1.000]	0.989 [0.968, 1.000]	0.162 [0.143, 0.184]	1.000 [1.000, 1.000]	0.985 [0.962, 1.000]

Strategy	P@5	P@10	R@10	MRR	NDCG@10
Hybrid, two-stage SQL, k = 500	0.986 [0.957, 1.000]	0.989 [0.968, 1.000]	0.162 [0.143, 0.182]	1.000 [1.000, 1.000]	0.985 [0.962, 1.000]

The two-stage query returned the same top-10 results as the single-stage query for candidate_k values from 50 to 500 on the 560-posting evaluation pool. Candidate_k = 100 is the production default. This result applies to that pool; on the 100,000-row performance benchmark, top-10 overlap was 68.2%. The vector-weight sweep is reported in Table 7.12.

Table 7.12 Sweep of the Vector Weight α

α	P@5	P@10	R@10	MRR	NDCG@10
0.0	0.921	0.889	0.148	0.976	0.896
0.1	1.000	1.000	0.164	1.000	1.000
0.2	1.000	1.000	0.164	1.000	1.000
0.3	1.000	1.000	0.164	1.000	1.000
0.4	1.000	1.000	0.164	1.000	1.000
0.5	1.000	1.000	0.164	1.000	1.000
0.6	1.000	1.000	0.164	1.000	0.997
0.7	0.986	0.989	0.162	1.000	0.985
0.8	0.979	0.971	0.159	0.976	0.964
0.9	0.950	0.946	0.155	0.973	0.943
1.0	0.936	0.932	0.153	0.945	0.923

On the synthetic retrieval benchmark, the production hybrid achieved P@5 = 0.986 and NDCG@10 = 0.985, compared with 0.921 and 0.896 for keyword-only matching and 0.936 and 0.923 for semantic-only matching. The hybrid also placed a relevant posting first for every profile (MRR = 1.000). The benchmark deliberately included paraphrased postings and keyword distractors, so these results show performance on those constructed cases; they do not establish the same quality on real job postings. The weight α = 0.7 was chosen before this evaluation and was not tuned on its labels. Lower vector weights from 0.1 to 0.5 scored higher on this dataset. Recall@10 remained low because each profile had many relevant postings but only ten results were evaluated.

7.5.6 Generation Quality

The factual grounding of generated text was evaluated for unsloth/Llama-3.2-3B-Instruct and TinyLlama-1.1B-Chat on 28 resume–job pairs each (56 outputs), on the CPU with 8 threads. The prompt instructed the model to use only the candidate's actual history, and a constrained post-processing step forces the output into a fixed format: a Summary section followed by exactly three bullet points. Format compliance of 28 of 28 for both models is therefore a property of the post-processing and not a measure of the models. The three bullets of each output (84 per model) were checked automatically, and a bullet that introduced a framework, tool or library absent from the candidate's resume was counted as an unsupported claim. Table 7.13 gives the results.

Table 7.13 Generation Quality and Grounding

Metric	Llama-3.2-3B-Instruct	TinyLlama-1.1B-Chat
Mean latency per generation (CPU, 8 threads)	54.42 s	20.63 s
Mean throughput	1.7 tokens/s	3.2 tokens/s
Mean length	72.8 words	51.5 words
Outputs in the required format	28 of 28 (enforced)	28 of 28 (enforced)
Bullets checked	84	84
Unsupported claims found	8 (9.5% of bullets)	25 (29.8% of bullets)
Mean qualitative score (10 outputs per model, scale 1 to 5)	4.20	2.75

Twenty of the 56 generated outputs (10 per model) were also scored on a five-point scale using the review sheet at evaluation/results/generation_samples_review.csv. The mean score was 4.20 for Llama-3.2-3B-Instruct and 2.75 for TinyLlama-1.1B-Chat. This agrees with the automatic technology check, which found fewer unsupported technology mentions in the larger model's output. The qualitative review was a single pass, with the automatic flags visible, so the scores are indicative rather than independent or blind. The automated check only flags technologies missing from the resume; it does not detect every possible unsupported detail, such as dates, employers or numerical claims. Generation latency also depends on prompt length.

7.5.7 Scope and Limits of the Evaluation

The evaluation covers correctness checks, retrieval latency, memory use, retrieval ranking and the grounding of generated text. However, the vector benchmark uses replicated descriptions, the retrieval benchmark uses synthetic profiles and programmatic labels, the qualitative review covers only 20 outputs and was not blind, the sequential-scan latency varied between runs, and GPU generation was not measured. These limitations are considered again in Sections 8.3 and 8.6.

7.6 Performance Metrics

Table 7.14 consolidates the measurements reported in this chapter.

Table 7.14 Summary of Measured Metrics

Metric	Value
HNSW query latency, 100,000 rows (m = 16, ef_construction = 128, ef_search = 400)	2.88 ms mean (p50 2.70 ms; p95 4.59 ms; p99 6.69 ms); recall@10 0.9565
Exact sequential scan, 100,000 rows	41.33 ms in the sweep run; 28.68 to 29.79 ms in three repeats; HNSW is about 10 to 14 times faster
HNSW query latency, 10,000 rows (ef_search = 400)	1.76 ms mean; recall@10 1.0000; sequential scan 4.17 ms
HNSW recall@10 at the default ef_search = 40	0.8795 (10,000 rows); 0.5800 (100,000 rows)
Hybrid query in the benchmark script, single-stage versus two-stage	44.55 ms versus 1.35 ms (10,000 rows); 130.60 ms versus 2.45 ms (100,000 rows)
POST /jobs/matches end to end, two-stage versus single-stage	8.43 ms versus 90.01 ms (10,000 rows); 9.08 ms versus 890.88 ms (100,000 rows); p95 at 100,000 rows 11.13 ms
Generation cache, full CPU mode	61.33 s cold, 2.00 ms cached; 30,637 times faster
CPU generation throughput	1.82 tokens per second (10 threads); 2.10 tokens per second (4 threads)
Peak backend memory (RSS)	7,225.99 MB (7.06 GB); baseline 15.94 MB; 875.93 MB after the embedding model
Offline operation	Passed; embedding and hybrid matching 145.55 ms
Automated tests	126 passed, 0 failed, 14.60 s
Backend line coverage	79% overall

Metric	Value
Retrieval quality of the production hybrid (hard synthetic benchmark)	P@5 0.986; P@10 0.989; R@10 0.162; MRR 1.000; NDCG@10 0.985; two-stage query identical to single-stage for k = 50 to 500
Grounding check of generated text (84 bullets per model)	8 unsupported claims for Llama-3.2-3B (9.5%); 25 for TinyLlama (29.8%); mean qualitative score 4.20 and 2.75

7.7 Comparative Analysis

Table 7.15 compares the baseline and optimised configurations that were measured. For the comparison with other commercial tools, see Table 2.1 in Chapter 2.

Table 7.15 Comparison of Baseline and Optimised Configurations

Aspect	Baseline	Optimised	Effect
Vector search, 10,000 rows	Sequential scan, 4.17 ms	HNSW (ef_search = 400), 1.76 ms	2.4 times faster; recall 1.0000
Vector search, 100,000 rows	Sequential scan, 29 to 41 ms	HNSW (ef_search = 400), 2.88 ms	10 to 14 times faster; recall 0.9565
Hybrid ranking through the API, 100,000 rows	Single-stage query, 890.88 ms	Two-stage query, 9.08 ms	98.1 times faster through the API on the 100,000-row benchmark; 100% top-10 overlap was measured separately on the 560-posting retrieval pool
Repeated AI generation (CPU)	Cold generation, 61.33 s	Cache hit, 2.00 ms	30,637 times faster
Ranking strategy (hard synthetic benchmark)	Keyword-only: P@5 0.921, NDCG@10 0.896	Hybrid 0.7: P@5 0.986, NDCG@10 0.985	Hybrid ranks better on this benchmark
Generation model (grounding)	TinyLlama: 29.8% unsupported bullets, score 2.75	Llama-3.2-3B: 9.5% unsupported bullets, score 4.20	Larger model fabricates fewer technologies, but is 2.6 times slower

Chapter 8 - Results and Conclusion

8.1 Results

Job Scout combines job discovery, resume-to-job matching, ATS keyword analysis, local AI-assisted writing and application tracking in one self-hosted application. The design addresses practical limits of workflows that depend on exact keyword overlap or separate tools, while keeping the core processing on the user's machine.

The measurements in Chapter 7 show that pgvector HNSW queries over 100,000 rows averaged 2.88 ms with Recall@10 of 0.9565 at ef_search = 400. The query was about 10 to 14 times faster than the measured sequential-scan baselines, although recall fell to 0.5800 at the default ef_search of 40. The application therefore sets the parameter explicitly. The single-stage hybrid query averaged 890.88 ms through the API at 100,000 rows, while the two-stage query averaged 9.08 ms; its 95th-percentile latency was 11.13 ms. The two-stage and single-stage query returned the same top-10 results on the 560-posting evaluation pool, but their overlap was 68.2% on the separate 100,000-row performance benchmark. The local Llama-3.2-3B-Instruct model generated text on the tested CPU-only laptop at about 1.7 to 2.1 tokens per second and reached a peak backend memory footprint of 7.06 GB. The response cache returned repeated requests in 2.00 ms compared with 61.33 seconds for cold generation. The offline audit completed the tested parsing, matching and generation workflow with external network access blocked. The automated suite passed 126 tests, with 79% backend code coverage. On the synthetic retrieval benchmark, the production hybrid achieved P@5 = 0.986, NDCG@10 = 0.985 and MRR = 1.000, compared with P@5 = 0.921 and NDCG@10 = 0.896 for keyword-only matching and P@5 = 0.936 and NDCG@10 = 0.923 for semantic-only matching. The grounding check found unsupported technology mentions in 9.5% of the larger model's generated bullets and 29.8% of TinyLlama's bullets.

Table 8.1 relates the outcome to each objective stated in Section 1.3.

Table 8.1 Objectives and Outcomes

Objective	Outcome
To build an automated pipeline for collecting job listings using Playwright and Celery-Beat scheduling.	Implemented: the report documents five scraper sources and a 10-minute Celery-Beat schedule. URL-based duplicate skipping was verified by integration test INT-02.
To develop a hybrid job-matching engine using pgvector, with a target mean query latency below 10 ms on the tested 100,000-row benchmark.	Implemented with qualifications: raw HNSW vector queries averaged 2.88 ms at 100,000 rows (Recall@10 = 0.9565, ef_search = 400). The two-stage hybrid API query averaged 9.08 ms, meeting the mean target; p95 and p99 were 11.13 ms and 12.05 ms. Retrieval quality at 100,000 rows was not independently evaluated with relevance labels.
To integrate locally hosted language models through Hugging Face Transformers and PyTorch for drafting resume bullet points and cover letters.	Implemented and tested on CPU. Llama-3.2-3B-Instruct generated at 1.82 tokens/s in the 10-thread throughput test; a separate four-thread cold-generation benchmark averaged 2.10 tokens/s. TinyLlama was attempted as a fallback. The offline test passed for the defined parsing, embedding, matching and generation workflow while external network access was blocked; this is not an exhaustive proof of zero egress on every path. In the grounding check, 9.5% of Llama-generated bullets mentioned a technology absent from the resume.
To build a responsive web application with FastAPI for backend processing and Next.js for the frontend.	Implemented with a Next.js/React frontend and FastAPI backend, packaged in a six-service Docker Compose deployment. The application exposes 29 application routes plus four documentation routes. Responsive behaviour was not separately quantified in the reported tests.

8.2 Discussion and Analysis

On the tested 100,000-row dataset, the HNSW index made vector retrieval about 10 to 14 times faster than the measured sequential-scan baselines, with Recall@10 of 0.9565 at ef_search = 400. The default value of 40 produced Recall@10 of 0.5800 at the same row count. The application therefore uses an explicit query-time value. The build parameters m = 16 and ef_construction = 128 were selected from the measured sweep as a relatively low-cost configuration that met the recall target.

The query-plan analysis showed that the original single-stage hybrid query did not use the HNSW index to order results because its ORDER BY clause used a composite expression. The two-stage query retrieves approximate vector candidates first and then reranks that candidate set using the hybrid score. It was faster in both the query-script and API benchmarks. Because reranking is limited to the candidate set, the result can differ from the exact ranking: the top-10 overlap was 100% on the 560-posting pool and 68.2% on the 100,000-row performance benchmark. These experiments used different datasets, and retrieval quality at 100,000 rows was not independently assessed with relevance labels.

The literature reviewed in Chapter 2 describes neural person–job matching and dense retrieval methods [10, 11, 12]. The project's evaluation is consistent with the usefulness of combining semantic and lexical evidence on the synthetic benchmark. However, the benchmark was constructed with paraphrased listings and keyword distractors, and vector weights from 0.1 to 0.5 scored higher than the production setting of 0.7. The results therefore support further testing of hybrid retrieval, but they do not establish that the current weighting is optimal for real job searches.

The local language model generates application text without sending its prompt to a hosted AI inference service in the tested configuration. Parsed resume information provides context, but it does not guarantee factual output. In the grounding check, 9.5% of bullets generated by Llama-3.2-3B-Instruct mentioned a technology absent from the resume, compared with 29.8% for TinyLlama. Candidates should review all generated material before using it in an application.

8.3 Limitations

Job Scout supports only English-language resumes and job descriptions. The local 3-billion-parameter model's output varies; it was compared with TinyLlama, not commercial models. Scraping depends on third-party site structures, and the single-user design lacks concurrent-user support. NFR-03 is partially met: TinyLlama is attempted if the primary model fails to load, while regex extraction handles resume parsing if both models fail; equivalent generation fallback is unavailable for every endpoint. Offline testing covered the defined parsing, embedding, matching and generation workflow with WAN egress blocked; it does not prove zero egress across every path or deployment.

- **Synthetic data and labels:** Vector-index performance was measured on 10,000- and 100,000-row datasets using replicated embeddings of real job text with small Gaussian noise. The separate API benchmark was built from 2,000 distinct generated job descriptions. Retrieval quality was evaluated on 28 synthetic candidate profiles and 560 job listings with programmatically assigned labels; no human-labelled evaluation on real postings was carried out.
- Latency target: the two-stage query meets the mean-latency target in NFR-01 at 100,000 rows (9.08 ms through the API). Its 95th and 99th percentiles were 11.13 ms and 12.05 ms, respectively; these tail latencies are reported separately. Sequential-scan measurements varied from 28.68 ms to 41.33 ms across the reported runs.
- **Two-stage hybrid query:** the top-10 overlap with the single-stage ranking was 100% on the 560-posting pool but 68.2% on the 100,000-row benchmark, and the retrieval quality of the two-stage query at large scale was not measured.
- **Generation quality:** the format of the generated text is enforced by post-processing, the automatic check detects only invented technologies, the qualitative scoring covers 20 outputs and was done in a single pass with the automatic flags visible, so it is not blind, and GPU generation was not measured.
- **Retrieval weights:** the production weight $\alpha = 0.7$ was not tuned, and lower values performed better on the benchmark.
- **Scraping:** the scrapers depend on the structure of third-party websites and must be used in accordance with each portal's terms of service and robots policies; the system is intended for personal, low-volume use.

8.4 Lessons Learnt

Developing Job Scout highlighted the value of containerisation for reproducible deployment, the trade-offs between model size and inference speed on CPU hardware, and the need for a fallback path when a preferred model cannot be loaded.

The evaluation also showed why performance claims should be checked against query plans, recall and result quality as well as latency. At the default setting, the HNSW index returned only 58% of the exact top-10 neighbours on the 100,000-row test. The original

hybrid query bypassed the index. An early retrieval evaluation used labels derived from the same features as keyword ranking and was discarded, and the sequential-scan baseline varied across runs. Recording these limitations made the final results more useful and easier to interpret.

8.5 Conclusion

Job Scout brings job discovery, resume-to-job matching, ATS keyword analysis, local AI-assisted writing and application tracking into one self-hosted application. In the recorded tests, raw vector queries over 100,000 postings averaged 2.88 ms, and the two-stage hybrid query averaged 9.08 ms through the API. Hybrid ranking performed better than keyword-only ranking on the synthetic retrieval benchmark. The automated suite passed 126 tests with 79% backend coverage, and the offline test confirmed that the tested workflow could run while external network access was blocked. The evaluation also identified limits: retrieval labels were synthetic, the 95th and 99th percentile API latencies exceeded 10 ms, and some generated bullets included technologies absent from the source resume. Job Scout is a working prototype. Its ranking quality and generated content need further testing with real data, and users should verify generated material before including it in applications.

8.6 Future Enhancements

Several improvements would make Job Scout more useful and reliable in future versions:

- A browser extension could let users save a job listing from a supported webpage without requiring a separate scraper for every site.
- A future version could separate resume checks, cover letter editing, and interview preparation into distinct steps. This would need evaluation before adding a multi-agent framework such as LangChain or CrewAI.
- If the project needs to collect more listings, its scraping jobs could be distributed across workers with appropriate rate limits and in line with each website’s terms of service. The current project is intended for personal, low-volume use.
- Form filling could be explored as a later feature, with user confirmation before any application is submitted to an external recruitment system.
- The generation component could be evaluated with domain-specific fine-tuning, including LoRA, if a suitable and properly licensed training dataset becomes available.

- **Large-Scale Retrieval Quality:** Evaluate the two-stage query against the single-stage ranking on a labelled benchmark of the order of 100,000 postings, and tune candidate_k and the vector weight α on a held-out part.
- **Real-Data Evaluation:** Build a human-labelled set of resume–job pairs from real postings, repeat the evaluation of Table 7.11 on it, and replace the single-pass scoring of generated text by a blind review with several raters.
- **Generation Quality:** Extend the grounding check to dates, employers and metrics, reduce the share of unsupported technologies in the generated bullets, and measure generation speed on a CUDA-capable GPU.
- **Tail Latency:** Reduce the 95th and 99th percentile latency of POST /jobs/matches below 10 ms.

Chapter 9 - Appendices

Appendix 1 - Plagiarism Certificate

SmallSEOTools

Plagiarism Detection Report by SmallSEOTOOLS



Plagiarism	0%	Partial Match	0%
Exact Match	0%	Unique	100%

Scan details

Total Words	Total Characters	Plagiarized Sentences	Unique Sentences
286	2036	0	15 (100%)

#1 100% Unique

Job Scout is a self-hosted platform that helps users find job vacancies suited to their resumes, review each match, and prepare application materials.

The project addresses two practical issues. Keyword-based filters can miss relevant experience when candidates describe the same skill in different words. In addition, online resume tools may process personal documents on provider-operated systems, depending on the service and its settings. Job Scout aggregates job ads from five employment platforms utilizing Playwright and Celery. It converts resumes and job descriptions into embeddings with all-MiniLM-L6-v2 and stores and searches those vectors in PostgreSQL using pgvector and HNSW. The ranking combines semantic similarity (70%) with keyword matching (30%). An ATS diagnostic highlights missing keywords, and a locally hosted Llama-3.2-3B-Instruct model uses retrieval-augmented generation (RAG) to help draft resume bullet points and cover letters. FastAPI, Next.js, Redis and Docker Compose connect the API, interface, background tasks and deployment.

In the recorded CPU-only tests, raw vector queries over 100,000 postings averaged 2.88 ms, while the two-stage hybrid query averaged 9.08 ms through the API; its 95th-percentile latency was 11.13 ms. The automated suite passed 126 tests, with 79% backend code coverage. On the synthetic retrieval benchmark, hybrid ranking reached an NDCG@10 score of 0.985, compared with 0.896 for keyword-only ranking. The evaluation also found that 9.5% of bullets generated by Llama-3.2-3B-Instruct mentioned a technology absent from the resume. Because the retrieval labels were synthetic rather than human-labelled examples from real job searches, these measurements should be treated as an initial evaluation. The tests show that the main workflow can run locally; retrieval quality on real job data and the factual accuracy of generated content need further evaluation.

Keywords: Retrieval-Augmented Generation, Resume Parsing, Artificial Intelligence, Job Matching, Vector Search.

Powered by SmallSEOTools Plagiarism Checker Blog About Us Contact Us

Appendix 2 - Screenshots

Fig. 9.1 to Fig. 9.7 show the implemented Job Scout interface. The views correspond to the modules described in Section 6.1; wireframes of the layout are given in Section 5.7.

Fig. 9.1 shows the job discovery view, with the list of best matches, each card carrying a match percentage, and the active profile panel with the resume upload area.

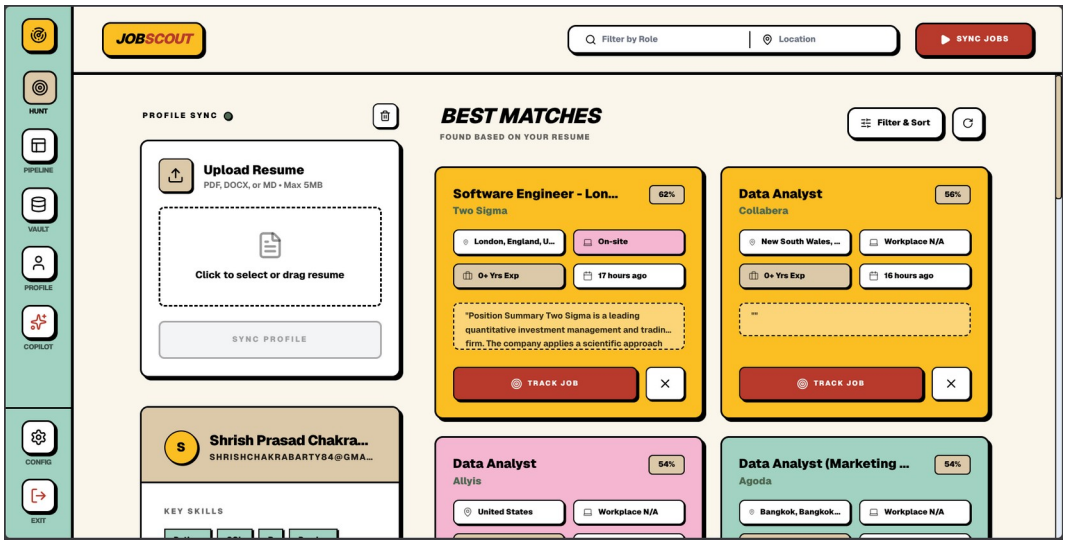


Fig. 9.1 Job Discovery View (Best Matches)

Fig. 9.2 shows the Resume Vault, which lists the stored resumes and provides the upload area used to add a new resume.



Fig. 9.2 Resume Vault

Fig. 9.3 shows the Job Tracker, the Kanban board on which saved jobs are organised into application stages.

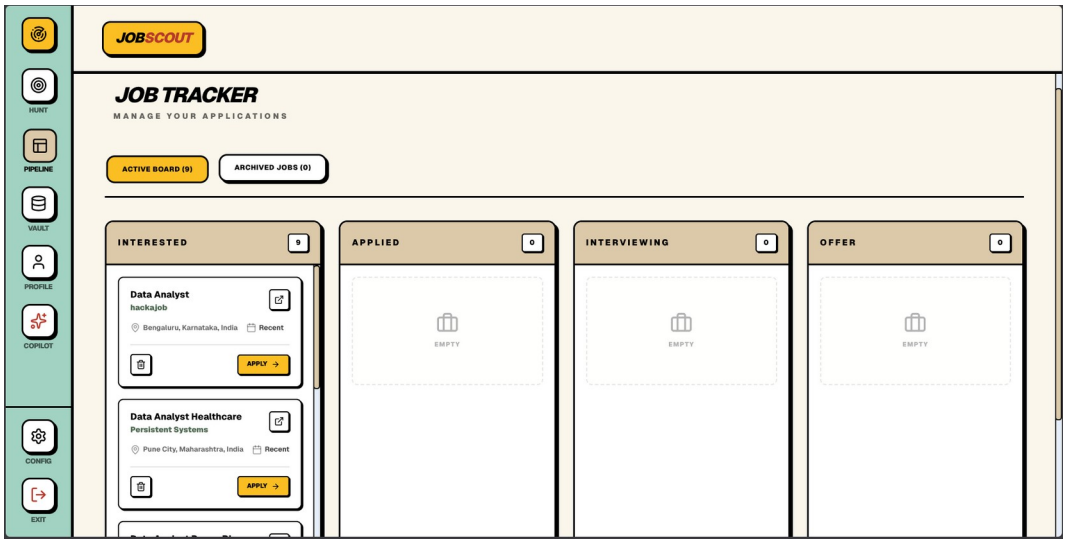


Fig. 9.3 Job Tracker (Kanban Board)

Fig. 9.4 shows the AI Copilot, in which the target resume and the local model parameters are selected before tailored content is generated.

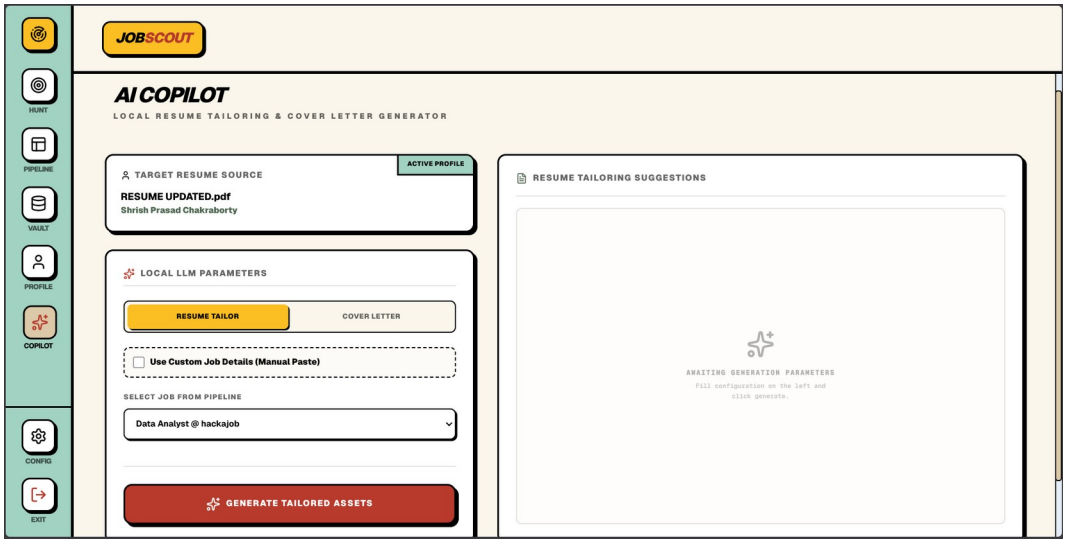


Fig. 9.4 AI Copilot View

Fig. 9.5 shows the Neural Profile view, which presents the skills and work history extracted from the active resume.

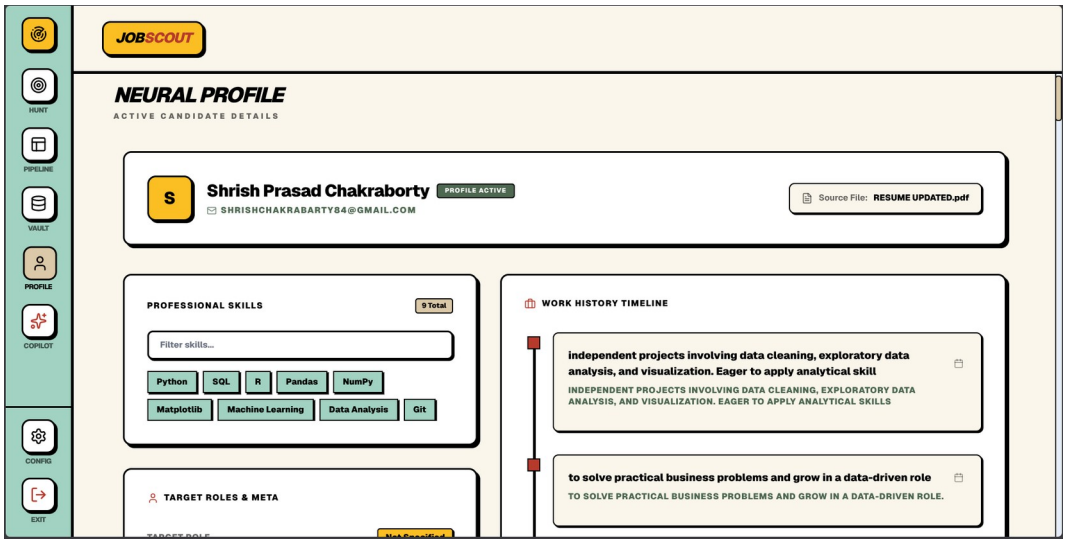


Fig. 9.5 Neural Profile View

Fig. 9.6 shows the Market Intelligence view (Market Radar), which summarises the scraped job data, including the distribution of experience requirements.

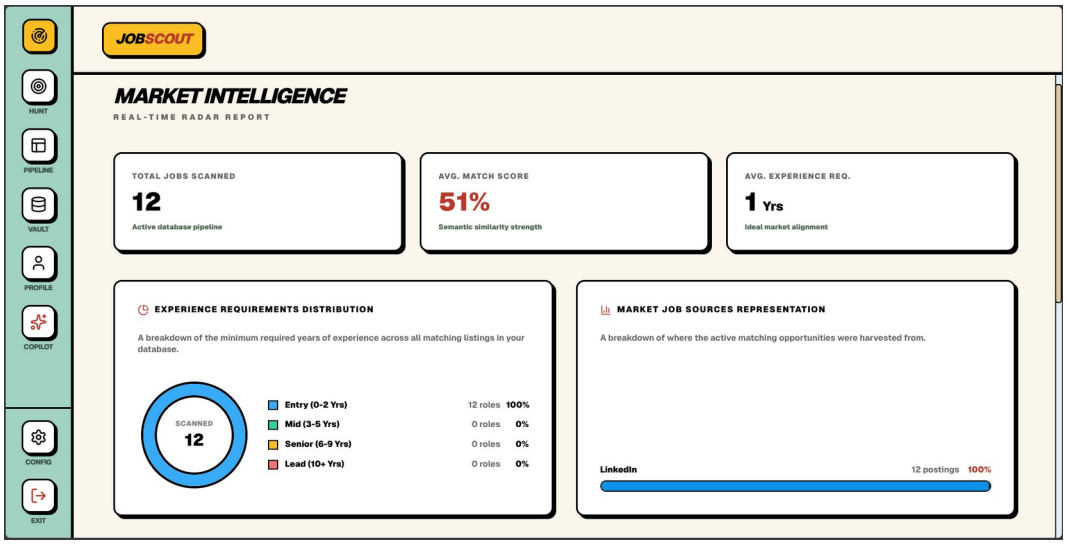


Fig. 9.6 Market Intelligence View (Market Radar)

Fig. 9.7 shows the System Configuration dialog, which provides the display and computation-device options and the controls for clearing the cache.

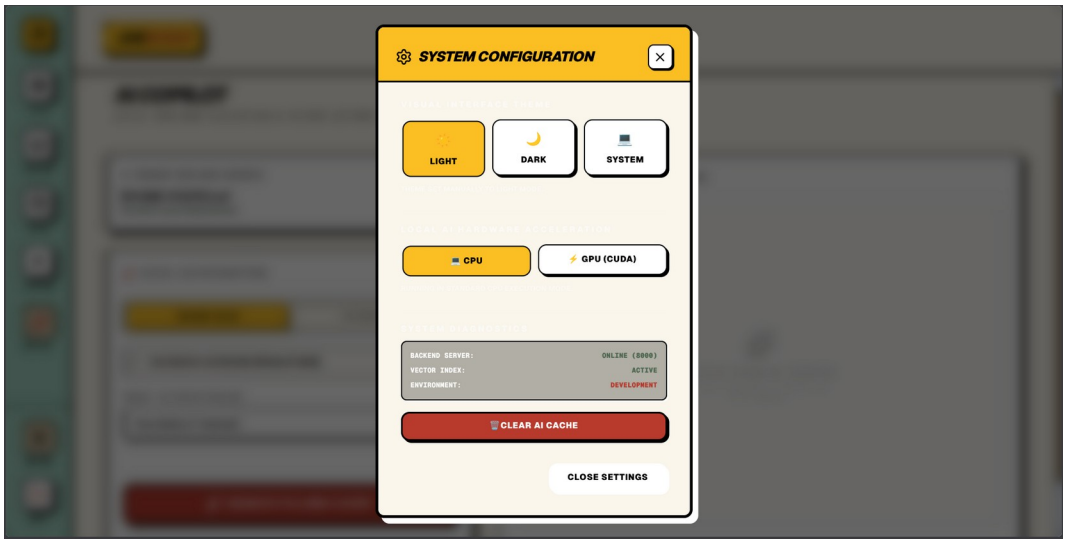


Fig. 9.7 System Configuration Dialog

Appendix 3 - Sample Coding

The project source code is publicly available in the repository at https://github.com/ShrishPC/Job-Scout. The repository archive reviewed for this report did not include a root-level licence file, so this report does not assign a repository-wide software licence. The Python and TypeScript source comprises approximately 9,500 lines, excluding dependencies and lock files. Table 9.1 lists key implementation files, and the listings below reproduce excerpts from the repository.

9.1 Key Implementation Files

Table 9.1 Key Implementation Files

File	Responsibility
backend/app/main.py	FastAPI application, request validation and all REST endpoints
backend/app/models/models.py	SQLAlchemy models, including the pgvector Vector(384) columns
backend/app/services/llm_service.py	Embedding generation, local LLM loading, RAG prompts and generation
backend/app/services/ matching_service.py	Hybrid pgvector and keyword job matching query
backend/app/services/ats_service.py	ATS keyword diagnostic
backend/app/services/resume_service.py	Resume text extraction and Markdown conversion
backend/app/services/export_service.py	DOCX and PDF export of resumes and ATS reports
backend/migrations/ add_vector_index.sql	HNSW and supporting index definitions
scraper/tasks.py, celery_app.py	Celery tasks and the Celery-Beat schedule
scraper/*_scraper.py	Playwright and feed scrapers for the job portals
frontend/src/components/ui/	React components (Kanban, Vault, ATS modal, AI Copilot, Radar, Profile)
scripts/benchmark_pgvector.py	Benchmark of sequential scan versus HNSW on 10,000 mock jobs
tests/	Automated pytest suite (unit, integration and security tests)
scripts/benchmarks/	Benchmarks for pgvector latency and the HNSW parameter sweep, the LLM cache, generation speed, memory and offline operation

File	Responsibility
evaluation/	Labelled data and scripts for the retrieval evaluation
docs/benchmarks/results/	Raw JSON and Markdown results of the benchmarks
docker-compose.yml	Six-service Docker Compose deployment

9.2 Database Models

The listing shows backend/app/models/models.py.

```
from sqlalchemy import Column, Integer, String, JSON, Text, DateTime, Boolean
from sqlalchemy.sql import func
from sqlalchemy.ext.declarative import declarative_base
from pgvector.sqlalchemy import Vector

Base = declarative_base()

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, index=True)
    email = Column(String, unique=True, index=True)
    resume_markdown = Column(Text)
    parsed_data = Column(JSON)  # Stores skills, experience, etc.
    embedding = Column(Vector(384))  # 384 dimensions for local MiniLM embeddings
    created_at = Column(DateTime(timezone=True), server_default=func.now())

class Resume(Base):
    __tablename__ = "resumes"

    id = Column(Integer, primary_key=True, index=True)
    filename = Column(String)
    resume_markdown = Column(Text)
    parsed_data = Column(JSON)  # Stores skills, experience, etc.
    embedding = Column(Vector(384))  # 384 dimensions for local MiniLM embeddings
    is_active = Column(Boolean, default=False)
    created_at = Column(DateTime(timezone=True), server_default=func.now())

class Job(Base):
    __tablename__ = "jobs"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String, index=True)
    company = Column(String, index=True)
    description = Column(Text)
    location = Column(String)
    salary = Column(String)
    job_url = Column(String, unique=True)
    parsed_data = Column(JSON)
    embedding = Column(Vector(384))  # 384 dimensions for local MiniLM embeddings
    experience_required = Column(Integer, default=0)
    workplace_type = Column(String, default="unspecified") # remote, hybrid,
onsite, negotiable, unspecified
    date_posted = Column(String) # Captured from scraper
    created_at = Column(DateTime(timezone=True), server_default=func.now())
```

```
class UserJobMatch(Base):
    __tablename__ = "user_job_matches"

    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(Integer, index=True)
    job_id = Column(Integer, index=True)
    status = Column(String, default="interested") # interested, applied,
interviewing, offered, rejected
    notes = Column(Text)
    updated_at = Column(DateTime(timezone=True), onupdate=func.now())
    created_at = Column(DateTime(timezone=True), server_default=func.now())

class AIGenerationCache(Base):
    __tablename__ = "ai_generation_caches"

    id = Column(Integer, primary_key=True, index=True)
    cache_key = Column(String, unique=True, index=True)
    response_text = Column(Text)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
```

9.3 Index Definitions

The listing shows backend/migrations/add_vector_index.sql.

```
-- HNSW index for fast cosine similarity search on job embeddings
CREATE INDEX IF NOT EXISTS idx_jobs_embedding_hnsw ON jobs USING hnsw (embedding
vector_cosine_ops);

-- Supporting indexes
CREATE INDEX IF NOT EXISTS idx_resume_is_active ON resumes (is_active) WHERE
is_active = true;
CREATE INDEX IF NOT EXISTS idx_jobs_workplace_type_lower ON jobs
(LOWER(workplace_type));
CREATE INDEX IF NOT EXISTS idx_user_job_matches_composite ON user_job_matches
(user_id, job_id);
```

Appendix 4 - User Documentation

i) Installation Instructions

1. For the local launcher, install Docker with Docker Compose, Node.js 20 and Python 3.11. Make sure Docker is running before starting the application. If using the Docker-only workflow, follow the model preparation step below and check the repository instructions for host dependencies.
2. Clone the repository with ‘git clone https://github.com/ShrishPC/Job-Scout.git’ and open a terminal in the project root directory.
3. Start the system with the launcher: ./run.sh on Linux or macOS, or .\run.ps1 in Windows PowerShell. The launcher checks Docker, creates the .env file, prepares the virtual environments, installs dependencies, downloads the local AI models, installs the Playwright browser dependencies, and launches PostgreSQL, Redis, FastAPI, the Celery workers and the Next.js server.
4. For a Docker Compose-only setup, prepare the model files first by running python scripts/download_models.py. The containers set HF_HUB_OFFLINE=1 and TRANSFORMERS_OFFLINE=1, so the required model files must already be available locally. Then run docker compose up --build.
5. Open the frontend at http://localhost:3000 and the API documentation at http://localhost:8001/docs. Press Ctrl+C in the launcher terminal to shut the services down cleanly.

ii) Using the Application

1. Navigate to http://localhost:3000 in a web browser.
2. Upload a resume (PDF, DOCX, DOC, MD or TXT; up to 10 MB) in the Resume Vault. The system parses the resume and generates its vector embedding.
3. Open the job discovery view to see the semantically matched job listings, and use the search and workplace-type filters to narrow them.
4. Open any job to run the ATS diagnostic, or open the AI Copilot to generate a tailored resume or cover letter.
5. Move saved jobs across the Kanban board to track the status of each application.
6. Use the system configuration panel to switch between CPU, GPU and demo mode, to clear the AI cache, and to shut down or restart the stack.

Appendix 5 - Glossary

Table 9.2 Glossary of Terms

Term	Meaning
ATS	Applicant Tracking System; automated software used by employers to filter job applications
ANN	Approximate Nearest Neighbour; search that trades a small loss in accuracy for large gains in speed
BERT	Bidirectional Encoder Representations from Transformers; a pre-trained language model
bfloat16	A 16-bit brain floating-point format that keeps the exponent range of 32-bit floats while halving memory use
Celery	A distributed task queue for Python
Cosine similarity	A measure of the angle between two vectors, used to compare embeddings
Embedding	A dense numeric vector that represents the meaning of a text
FastAPI	A modern Python web framework for building APIs
HNSW	Hierarchical Navigable Small World; a graph-based algorithm for approximate nearest neighbour search
Kanban	A board that tracks items across stages such as Interested, Applied and Interviewing
LLM	Large Language Model
pgvector	A PostgreSQL extension that adds vector similarity search
MRR	Mean Reciprocal Rank; an evaluation metric based on the rank of the first correct result
Playwright	A browser automation framework used for scraping JavaScript-rendered pages
RAG	Retrieval-Augmented Generation; a technique that grounds LLM output in retrieved context
SBERT	Sentence-BERT; a BERT variant that produces sentence embeddings suited to cosine comparison
HTTP streaming	Sending a response in chunks so the client can display generated text as it is produced

Chapter 10 - References

[1] Raghavan, Manish, Solon Barocas, Jon Kleinberg, and Karen Levy. "Mitigating Bias in Algorithmic Hiring: Evaluating Claims and Practices." Proceedings of the 2020 ACM Conference on Fairness, Accountability, and Transparency, 2020, pp. 469–481.

[2] Carlini, Nicholas, et al. "Extracting Training Data from Large Language Models." Proceedings of the 30th USENIX Security Symposium, 2021, pp. 2633–2650.

[3] Jobscan. “2026 Applicant Tracking System (ATS) Usage Report: Key Shifts and Strategies for Job Seekers.” Jobscan Blog, 2026. https://www.jobscan.co/blog/fortune-500-use-applicant-tracking-systems/. Accessed 12 October 2026.

[4] Bogen, Miranda, and Aaron Rieke. "Help Wanted: An Examination of Hiring Algorithms, Equity, and Bias." Upturn, 2018. https://www.upturn.org/work/help-wanted/. Accessed 10 October 2026.

[5] Fuller, Joseph B., Manjari Raman, Eva Sage-Gavin, and Kristen Hines. "Hidden Workers: Untapped Talent." Harvard Business School Project on Managing the Future of Work and Accenture, 2021. https://www.hbs.edu/managing-the-future-of-work/research/hidden-workers-untapped-talent. Accessed 13 October 2026.

[6] Deerwester, Scott, et al. "Indexing by Latent Semantic Analysis." Journal of the American Society for Information Science, vol. 41, no. 6, 1990, pp. 391–407.

[7] Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." Proceedings of NAACL-HLT, 2019, pp. 4171–4186.

[8] Reimers, Nils, and Iryna Gurevych. "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks." Proceedings of EMNLP-IJCNLP, 2019, pp. 3982–3992.

[9] Sentence Transformers. “all-MiniLM-L6-v2 Model Card.” Hugging Face Model Hub. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2. Accessed 10 October 2026.

[10] Qin, Chuan, et al. "Enhancing Person-Job Fit for Talent Recruitment: An Ability-aware Neural Network Approach." Proceedings of the 41st International ACM SIGIR Conference on Research and Development in Information Retrieval, 2018, pp. 25–34. https://doi.org/10.1145/3209978.3210025.

[11] Qin, Chuan, et al. "An Enhanced Neural Network Approach to Person-Job Fit in Talent Recruitment." ACM Transactions on Information Systems, vol. 38, no. 2, 2020, Article 15. https://doi.org/10.1145/3376927.

[12] Karpukhin, Vladimir, et al. "Dense Passage Retrieval for Open-Domain Question Answering." Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, 2020, pp. 6769–6781.

[13] Malkov, Yu. A., and D. A. Yashunin. "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs." IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 42, no. 4, 2020, pp. 824–836.

[14] Aumüller, Martin, Erik Bernhardsson, and Alexander Faithfull. "ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms." Information Systems, vol. 87, 2020, Article 101374. https://doi.org/10.1016/j.is.2019.02.006.

[15] Kane, Andrew. pgvector: Open-Source Vector Similarity Search for Postgres. GitHub, 2021. https://github.com/pgvector/pgvector. Accessed 10 October 2026.

[16] Lewis, Patrick, et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474.

[17] Shuster, Kurt, et al. "Retrieval Augmentation Reduces Hallucination in Conversation." Findings of the Association for Computational Linguistics: EMNLP 2021, 2021, pp. 3784–3803.

[18] Gao, Yunfan, et al. "Retrieval-Augmented Generation for Large Language Models: A Survey." arXiv preprint arXiv:2312.10997, 2023.

[19] Izacard, Gautier, and Edouard Grave. "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering." Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics, 2021, pp. 874–880.

[20] Touvron, Hugo, et al. “Llama 2: Open Foundation and Fine-Tuned Chat Models.” arXiv preprint arXiv:2307.09288, 2023.

[21] Grattafiori, Aaron, et al. “The Llama 3 Herd of Models.” arXiv preprint arXiv:2407.21783, 2024.

[22] Jiang, Albert Q., et al. "Mistral 7B." arXiv preprint arXiv:2310.06825, 2023.

[23] Gemma Team, Google DeepMind. "Gemma: Open Models Based on Gemini Research and Technology." arXiv preprint arXiv:2403.08295, 2024.

[24] Meta AI. "Llama 3.2: Revolutionizing Edge AI and Vision with Open, Customizable Models." Meta AI Blog, 2024. https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/. Accessed 10 October 2026.

[25] Wolf, Thomas, et al. "Transformers: State-of-the-Art Natural Language Processing." Proceedings of EMNLP 2020: System Demonstrations, 2020, pp. 38–45.

[26] Gerganov, Georgi, and contributors. llama.cpp. GitHub, 2023. https://github.com/ggml-org/llama.cpp. Accessed 10 October 2026.

[27] Kwon, Woosuk, et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention." Proceedings of the 29th ACM Symposium on Operating Systems Principles, 2023, pp. 611–626.

[28] Celery Project. "Celery: Distributed Task Queue." Celery Documentation, 2026. https://docs.celeryq.dev/. Accessed 10 October 2026.

[29] Microsoft. "Playwright Documentation." Playwright, 2026. https://playwright.dev/. Accessed 10 October 2026.

[30] Teal. “Getting Started: Resume Builder.” Teal Help Center, 27 July 2026. https://help.tealhq.com/en/articles/9508933-getting-started-resume-builder. Accessed 12 October 2026.

[31] Rezi. “The #1 AI Resume Builder.” Rezi. https://www.rezi.ai/. Accessed 12 October 2026.

[32] Docker Inc. "Docker Documentation." Docker, 2026. https://docs.docker.com/. Accessed 10 October 2026.

[33] FastAPI. "FastAPI Documentation." FastAPI, 2026. https://fastapi.tiangolo.com/. Accessed 10 October 2026.

[34] Hugging Face. "Transformers Documentation." Hugging Face, 2026. https://huggingface.co/docs/transformers. Accessed 10 October 2026.

[35] Next.js. "Next.js Documentation." Vercel, 2026. https://nextjs.org/docs. Accessed 10 October 2026.

[36] PostgreSQL Global Development Group. "PostgreSQL Documentation." PostgreSQL, 2026. https://www.postgresql.org/docs/. Accessed 10 October 2026.

[37] PyTorch. "PyTorch Documentation." The Linux Foundation, 2026. https://pytorch.org/docs/. Accessed 10 October 2026.

[38] Redis. "Redis Documentation." Redis Ltd., 2026. https://redis.io/docs/. Accessed 10 October 2026.

[39] Hugging Face. “unsloth/Llama-3.2-3B-Instruct Model Card.” Hugging Face Model Hub. https://huggingface.co/unsloth/Llama-3.2-3B-Instruct. Accessed 12 October 2026.

[40] Zhang, Peiyuan, et al. “TinyLlama: An Open-Source Small Language Model.” arXiv preprint arXiv:2401.02385, 2024. https://arxiv.org/abs/2401.02385.